

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Sztricskó Zoltán Milán

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**Mesterséges Intelligencia alapú diákasszisztens
webalkalmazás matematikai feladatok
támogatására**

Szakedolgozat

Készítette:

Sztricskó Zoltán Milán
programtervező
informatikus szakos
hallgató

Témavezető:

Dr. Biliczki Vilmos
egyetemi adjunktus

**Szeged
2026**

Feladatkiírás

A matematikai feladatok önálló gyakorlása során a tanulók gyakran nem teljes megoldást, hanem érthető, lépésenkénti segítséget igényelnek. A feladat egy olyan webalkalmazás elkészítése, amely mesterséges intelligencia segítségével támogatja a matematikai példák megértését, gyakorlását és ellenőrzését.

A rendszerben a felhasználók regisztráció után be tudnak jelentkezni, matematikai kérdéseket vagy feladatokat küldhetnek be, valamint új gyakorló feladatokat kérhetnek. Az alkalmazás célja, hogy a tanulási folyamatot segítse. Ne minden esetben adjon azonnal teljes megoldást, hanem rövid magyarázatokkal, következő lépésekkel és visszajelzésekkel vezesse a felhasználót a megoldás felé.

A webalkalmazásnak tartalmaznia kell a korábbi beszélgetések mentését és folytatását, hogy a felhasználó később is visszatérhessen a megkezdett feladataihoz. A rendszernek kezelnie kell a nem matematikai jellegű kéréseket is és ezekre megfelelő elutasító választ kell adnia.

A megvalósítás során frontend, backend és adatbázis komponensek készülnek. A backend feladata a hitelesítés, az adatok kezelése, a beszélgetések tárolása és az AI alapú tutor modul működtetése. A frontend egy áttekinthető, reszponzív felületet biztosít a felhasználó számára. A dolgozatban be kell mutatni a rendszer tervezését, működését, az alkalmazott technológiákat, valamint a tesztelés és validáció eredményeit.

Tartalmi összefoglaló

Téma megnevezése:

Mesterséges intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

A megadott feladat megfogalmazása:

A feladat egy olyan webalkalmazás elkészítése, amely matematikai feladatok gyakorlását AI alapú, lépésenkénti segítséggel támogatja. A rendszer kezeli a felhasználói fiókokat, menti a beszélgetéseket és kiszűri a nem matematikai jellegű kéréseket.

Megoldási mód:

A megoldás frontendből, backendből és adatbázisból álló webalkalmazásként készült el, Visual Studio Code^[1] fejlesztői környezetben.

Alkalmazott eszközök, módszerek:

A frontend Next.js^[2] és TypeScript^[3] használatával készült, a backend FastAPI^[4] alapú, az adatok tárolását PostgreSQL^[5] biztosítja. A futtatási környezet összeállítását Docker^[6] segíti. Az AI funkciókhoz Qwen alapú nyelvi modell^[7], valamint Python^[8] és kapcsolódó Python könyvtárak kerültek felhasználásra.

Elért eredmények:

Az elkészült alkalmazás támogatja a regisztrációt, a bejelentkezést, valamint a matematikai beszélgetések indítását és folytatását, valamint a korábbi beszélgetések mentését. A projekt forráskódja GitHubon publikusan elérhető.

Kulcsszavak:

mesterséges intelligencia, webalkalmazás, matematikai tutor, Next.js^[2], FastAPI^[4], PostgreSQL^[5]

Tartalomjegyzék

Feladatkiírás	1
Tartalmi összefoglaló	2
Tartalomjegyzék	3
Bevezetés	5
1. Kapcsolódó megoldások áttekintése	7
1.1 AI alapú tanulástámogató rendszerek	7
1.2 Matematikai feladatmegoldó alkalmazások	8
1.3 A vizsgált megoldások összehasonlítása	10
1.4 Saját alkalmazás helye a megoldások között	11
2. A rendszer követelményei és funkcionális specifikációja	12
2.1 Célcsoport és felhasználói szerepkörök	12
2.2 A rendszer célja és hatóköre	13
2.3 Funkcionális követelmények	14
2.3.1 Felhasználói fiókkezelés	14
2.3.2 Matematikai tutor beszélgetések kezelése	15
2.3.3 Beszélgetési előzmények és mappakezelés	15
2.3.4 Profilhoz kapcsolódó műveletek	16
2.3.5 Nem matematikai kérések kezelése	16
2.4 Nem funkcionális követelmények	17
2.5 Felhasználói folyamatok	18
2.5.1 Regisztráció és bejelentkezés folyamata	19
2.5.2 Beszélgetés indítása és folytatása	19
2.5.3 Mentett beszélgetések és mappák kezelése	20
2.5.4 Profilkezelés és profilműveletek	21
3. Felhasznált technológiák	22
3.1 Frontend technológiák	22
3.2 Backend technológiák	23
3.3 Adatbázis és konténerizáció	24
3.4 AI modell és Python könyvtárak	26
3.5 Fejlesztői környezet és verziókezelés	28
4. Rendszerterv	29
4.1 A rendszer architektúrája	29
4.2 Frontend, backend és adatbázis kapcsolata	30
4.3 Adatmodell	31
4.4 API-végpontok áttekintése	33
4.5 AI tutor integráció	34
5. Megvalósítás	36
5.1 Projektstruktúra	36
5.2 Felhasználókezelés megvalósítása	37
5.3 Chatfunkció és beszélgetések kezelése	38
5.4 Mappakezelés és profilfunkciók megvalósítása	39
5.5 AI tutor működése	41
5.6 Hibakezelés és ismert korlátok	42

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok
támogatására

6. Tesztelés és validáció	45
6.1 Backend és API tesztelése	45
6.2 Frontend és felhasználói működés ellenőrzése	46
6.3 AI tutor működésének validációja	47
7. Mesterséges intelligencia használata a fejlesztés során	50
8. Összegzés és továbbfejlesztés	52
Irodalomjegyzék	54
Nyilatkozat	55
Köszönetnyilvánítás	56
Elektronikus mellékletek	57
Mellékletek	58

Bevezetés

Az utóbbi években a digitális tanulástámogató eszközök egyre nagyobb szerepet kaptak a diákok és hallgatók mindennapi tanulásában. Az online tananyagok, webalkalmazások és mesterséges intelligencia alapú rendszerek lehetőséget adnak arra, hogy a tanulók önállóan is segítséget kapjanak egy-egy feladat megértéséhez vagy gyakorlásához. Ez különösen akkor lehet hasznos, amikor a tanuló otthon készül és nincs közvetlenül mellette tanár vagy más segítő személy.

A matematika tanulása ebből a szempontból különösen fontos terület, mert a feladatok megoldása nem csupán a végeredmény megtalálásáról szól. A tanulónak meg kell értenie a megoldáshoz vezető lépéseket, az alkalmazott szabályokat és azt is, hogy egy adott feladatnál miért az adott módszer vezet eredményre. A PISA 2022 mérés alapján a magyar tanulók matematikából 473 pontot értek el, ami gyakorlatilag megegyezik az OECD-országok 472 pontos átlagával^[9]. Ez alapján a magyar eredmény nemzetközi összehasonlításban átlag körüli, ugyanakkor a matematikai kompetenciák fejlesztése továbbra is fontos oktatási terület.

A mesterséges intelligencia alapú rendszerek alkalmasak lehetnek arra, hogy természetes nyelven feldolgozzák a felhasználó kérdéseit és azokra érthető választ adjanak. Oktatási célú felhasználásnál azonban fontos, hogy az alkalmazás ne csupán általános chatbotként működjön, hanem a tanulási folyamatot támogassa. Egy matematikai tutor jellegű rendszer célja ezért nem kizárólag a kész megoldás közlése, hanem a feladat megértésének segítése, a lépések irányítása és a tanuló válaszainak ellenőrzése.

A dolgozatomban bemutatott webalkalmazás ehhez a problémakörhöz kapcsolódik. A rendszer célja, hogy mesterséges intelligencia segítségével támogassa a matematikai feladatok gyakorlását és megértését. Az alkalmazásban a felhasználók regisztráció után be tudnak jelentkezni, matematikai kérdéseket vagy feladatokat küldhetnek be, valamint korábbi beszélgetéseiket is elérhetik és folytathatják.

Fontos szempont volt, hogy az alkalmazás a matematikai jellegű kérésekre fókuszáljon. Ennek megfelelően a rendszernek kezelnie kell azokat az eseteket is, amikor a felhasználó nem matematikai témájú kérdést tesz fel. Ilyenkor az

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

alkalmazásnak nem általános válasz generátorként kell működnie, hanem jeleznie kell, hogy csak matematikai kérdések és feladatok támogatására használható.

A témához kapcsolódóan több olyan digitális tanulástámogató és matematikai feladatmegoldó alkalmazás is elérhető, amely hasonló céllal segíti a tanulókat. Ezek a rendszerek eltérő módon támogatják a gyakorlást. Egyesek AI alapú párbeszédet biztosítanak, mások lépésenkénti magyarázatot, kamerás feladat felismerést vagy tanulókártyákat kínálnak. A következő fejezet ezeknek a kapcsolódó megoldásoknak a rövid áttekintésével foglalkozik, hogy látható legyen, milyen környezetbe illeszkedik a dolgozatban bemutatott webalkalmazás.

1. Kapcsolódó megoldások áttekintése

A fejezetben olyan meglévő alkalmazások és szolgáltatások kerülnek áttekintésre, amelyek funkcióik alapján kapcsolódnak a dolgozatban bemutatott rendszerhez. Az áttekintés nem teljes körű piaci elemzésként készült, hanem összehasonlító jelleggel mutatja be azokat a megoldásokat, amelyek valamilyen formában támogatják a tanulást, a feladatmegoldást vagy a felhasználóval folytatott interaktív párbeszédet.

A vizsgálat fő szempontjai közé tartozik, hogy az adott rendszer milyen típusú segítséget nyújt, mennyire interaktív a működése, biztosít-e lépésenkénti magyarázatot, valamint milyen mértékben kapcsolódik matematikai feladatok megoldásához. Ezek a szempontok segítenek abban, hogy a később bemutatott, általam készített alkalmazás funkciói és korlátai egyértelműbben értelmezhetők legyenek.

1.1 AI alapú tanulástámogató rendszerek

Ezen rendszerek közös jellemzője, hogy a felhasználóval természetes nyelven képesek kommunikálni. Ez azt jelenti, hogy a tanuló nem csak előre megadott gombok vagy menüpontok segítségével használhatja őket, hanem saját kérdéseket is megfogalmazhat. Ezek a rendszerek többféle tanulási helyzetben alkalmazhatók, például magyarázatok készítésére, feladatok értelmezésére, gyakorló kérdések létrehozására vagy tananyagok feldolgozására.

Az egyik ilyen megoldás a Khan Academy által fejlesztett Khanmigo. Ez egy AI alapú tanulási asszisztens, amely személyes tutorként és tanári segédeszközként is használható. A Khanmigo célja nem az, hogy egyszerűen megadja a kész választ, hanem az, hogy kérdésekkel és iránymutatással segítse a tanulót a megoldás megtalálásában. Ez a működés hasonlít egy tutor szemléletű rendszerhez, mert nem csak a végeredményre figyel, hanem a gondolkodási folyamat támogatására is. Előnye, hogy kifejezetten oktatási célra készült, hátránya viszont, hogy zárt szolgáltatásként működik, ezért egy saját fejlesztésű rendszerhez csak korlátozottan igazítható. [\[10\]](#)

Hasonló tanulástámogató megoldás a Quizlet AI is. A Quizlet eredetileg főként tanulókártyákról és gyakorló feladatokról ismert, az AI funkciók pedig ezt egészítik ki. A rendszer segíthet gyakorlótesztek, összefoglalók, tanulási segédletek és automatikusan létrehozott tananyagok készítésében. Ez a megközelítés főként az

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

ismétlést, a gyakorlást és a tananyag rendszerezését támogatja. Előnye, hogy sokféle tantárgyhoz használható, viszont nem kifejezetten matematikai tutor, ezért nem minden esetben a lépésenkénti feladatmegoldás áll a középpontban.^[11]

Az általánosabb AI alapú oktatási rendszerek közé sorolható a ChatGPT Edu is. Ez a ChatGPT felsőoktatási intézmények számára készült változata, amely természetes nyelvű párbeszédre épül. Felhasználható tanulási segítségként, kutatási támogatásként vagy akár oktatási és adminisztratív feladatok támogatására is. Előnye, hogy rugalmasan alkalmazható sokféle témában, ugyanakkor nem kizárólag matematikai feladatokra készült. Emiatt egy matematikai tutor alkalmazás esetében külön szabályokra és célzott működésre van szükség, hogy a rendszer valóban a matematikai tanulást támogassa.^[12]

A bemutatott rendszerek alapján látható, hogy az MI alapú tanulástámogatás többféle módon valósulhat meg. Vannak megoldások, amelyek a tanuló gondolkodását próbálják vezetni, míg más rendszerek inkább a tananyag feldolgozására, az ismételésre vagy a gyakorló feladatok létrehozására helyezik a hangsúlyt. A dolgozatban bemutatott alkalmazás ezekhez képest szűkebb területre fókuszál. Elsősorban matematikai kérdések és feladatok támogatására készült, miközben célja a rövid, lépésenkénti segítségnyújtás és a korábbi beszélgetések folytathatóságának biztosítása.

1.2 Matematikai feladatmegoldó alkalmazások

A tanulók gyakran találkoznak olyan matematikai feladatokkal, amelyeknél gyors segítségre vagy a megoldási lépések részletesebb magyarázatára van szükségük. Erre a problémára több olyan alkalmazás is készült, amely szövegesen megadott vagy képként feltöltött feladatokat dolgoz fel, majd bemutatja a megoldáshoz vezető lépéseket. A korábban bemutatott általánosabb AI alapú tanulástámogató rendszerekhez képest ezek az alkalmazások szűkebb területre fókuszálnak, mivel elsősorban matematikai számításokkal, egyenletekkel, algebrai átalakításokkal vagy más matematikai problémákkal foglalkoznak.

Az egyik legismertebb erre a problémára létrehozott alkalmazás a Photomath, amely kamerás feladat felismeréssel és szakaszos magyarázatokkal segíti a matematikai feladatok megértését. A felhasználó lefényképezheti vagy begépelheti a feladatot, az alkalmazás pedig megjeleníti a megoldást és annak főbb lépéseit. Előnye, hogy gyorsan használható és a fokozatos magyarázatok miatt nem csak a végeredményt jeleníti meg. Hátránya ugyanakkor, hogy a működés középpontjában sokszor a konkrét feladat

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

automatikus megoldása áll, nem pedig egy hosszabb, párbeszéd alapú tanulási folyamat.^[13]

Hasonló célú megoldás a Socratic by Google, amely tanulási segítséget nyújt különböző tantárgyakhoz, köztük matematikához is. A rendszerben a felhasználó kérdést adhat meg és az alkalmazás ehhez kapcsolódó magyarázatokat vagy tanulási tartalmakat jeleníthet meg. Előnye, hogy több tantárgyhoz is használható és egyszerű módon támogatja a felhasználót egy probléma megértésében. Ugyanakkor kevésbé egy saját, folyamatos beszélgetésre épülő matematikai tutor, hanem inkább olyan tanulási segéd, amely a kérdéshez kapcsolódó magyarázatokat és találatokat jelenít meg.^[14]

A Microsoft Math Solver szintén matematikai problémák megoldására készült. A rendszer lehetőséget ad matematikai feladatok megadására és irányított megoldási segítséget is nyújthat. A felhasználók többféle módon adhatnak meg feladatokat, például beírással vagy képalapú bevitellel. Előnye, hogy sok matematikai témakört támogat és gyors visszajelzést ad a felhasználónak. A dolgozatban bemutatott alkalmazáshoz képest azonban kevésbé a mentett párbeszéddek és a tutor jellegű beszélgetés áll a középpontban, hanem inkább a konkrét matematikai probléma megoldása.^[15]

A Wolfram Alpha más megközelítést képvisel, mivel számítási és tudásalapú rendszerként működik. Matematikai kérdések esetén képes különböző számításokat elvégezni, egyenleteket megoldani, függvényeket elemezni vagy grafikonokat megjeleníteni. Nagy előnye a széles matematikai és tudományos háttér, valamint az, hogy összetettebb számítások esetén is használható. Ugyanakkor a rendszer használata sok esetben inkább számítási eredmények lekérdezésére épül és kevésbé egy olyan tanulási párbeszédre, amely a tanulót lépésenként vezeti végig egy feladat megoldásán.^[16]

A bemutatott matematikai feladatmegoldó alkalmazások alapján látható, hogy ezeknek a rendszereknek az egyik legnagyobb erőssége a gyors feladatfeldolgozás és a megoldási lépések megjelenítése. Hasznosak lehetnek akkor, ha a felhasználó egy konkrét matematikai feladathoz keres segítséget. A dolgozatban bemutatott alkalmazás ezekhez képest nem kamerás feladat felismerésre vagy automatikus számítási motorra épül, hanem egy beszélgetésalapú webes felületre. A cél az, hogy a rendszer matematikai kérdések esetén tutor jellegű segítséget adjon és lehetővé tegye a korábbi beszélgetések mentését és folytatását.

1.3 A vizsgált megoldások összehasonlítása

A korábban bemutatott rendszerek eltérő módon támogatják a tanulást és a matematikai feladatmegoldást. Egyes alkalmazások főként MI alapú párbeszédre és tanulási segítségnyújtásra épülnek, míg mások konkrét matematikai feladatok felismerését, megoldását vagy magyarázatát helyezik előtérbe. Az összehasonlítás célja annak bemutatása, hogy ezek a rendszerek milyen funkciókkal rendelkeznek, milyen előnyöket kínálnak, valamint milyen korlátok figyelhetők meg a dolgozat témája szempontjából.

Megoldás	Használat	Fő előny	Fő hátrány
<i>Khanmigo</i>	MI-alapú tanulási asszisztens és tutorálás	Oktatási célra készült, irányított segítséget ad	Zárt rendszer, korlátozottan testreszabható
<i>Quizlet AI</i>	Tanulókártyák, gyakorlótesztek és ismétlés	Segít a tananyag rendszerezését és gyakorlását	Nem kifejezetten matematikai tutor
<i>ChatGPT Edu</i>	Általános MI-alapú asszisztens	Sokféle tanulási és oktatási helyzetben használható	Nem kizárólag matematikai feladatokra fókuszál
<i>Photomath</i>	Matematikai feladatok felismerése, megoldása	Gyors megoldást és magyarázatot ad	Inkább automatikus feladatmegoldásra épül
<i>Socratic by Google</i>	Tanulási segítség több tantárgyhoz	Egyszerű kérdésalapú használatot biztosít	Kevésbé párbeszédalapú matematikai tutor
<i>Microsoft Math Solver</i>	Matematikai problémák megoldása többféle bevitellel	Gyors visszajelzést ad, több témakört támogat	Főként konkrét feladatok megoldására fókuszál
<i>Wolfram Alpha</i>	Számítási és tudásalapú lekérdezések	Összetett matematikai és tudományos számításokra is alkalmas	Kevésbé tanulási párbeszédre épül

1.3.1. ábra: Összehasonlító táblázat

A táblázat alapján látható, hogy a vizsgált rendszerek többféle módon közelítik meg a tanulástámogatást. A Khanmigo, a Quizlet AI és a ChatGPT Edu inkább általánosabb AI-alapú tanulási környezetként értelmezhetők, míg a Photomath, a Socratic, a Microsoft Math Solver és a Wolfram Alpha erősebben kapcsolódnak matematikai problémák megoldásához vagy értelmezéséhez.

A dolgozat témája szempontjából fontos különbség, hogy a legtöbbet vizsgált rendszer vagy általános tanulástámogatást nyújt, vagy egy konkrét matematikai feladat gyors megoldását helyezi előtérbe. Ezzel szemben a dolgozatban bemutatott alkalmazás célja egy olyan webes prototípus megvalósítása, amely matematikai kérdésekre fókuszál, beszélgetésalapú felületen működik és lehetővé teszi a korábbi beszélgetések mentését s folytatását.

Az összehasonlítás alapján az is megfigyelhető, hogy a létező rendszerek több hasznos funkcióval rendelkeznek, azonban ezek nem minden esetben illeszkednek teljesen egy saját fejlesztésű, matematikai segédtanár jellegű alkalmazás céljaihoz. Ez indokolja egy olyan rendszer elkészítését, amely szűkebb területre koncentrálna és a matematikai tanulási folyamat támogatását helyezi középpontba.

1.4 Saját alkalmazás helye a megoldások között

Az előző alfejezetekben bemutatott példák jól mutatják, hogy a tanulástámogató rendszerek eltérő célokra és használati helyzetekre készülnek. Vannak olyan alkalmazások, amelyek szélesebb oktatási környezetben használhatók, míg mások egy-egy konkrét matematikai probléma megoldására adnak gyors segítséget. A saját fejlesztés szempontjából ezért fontos meghatározni, hogy az elkészített rendszer nem ezek teljes körű kiváltására törekszik, hanem egy szűkebb, jól körülhatárolható feladatot valósít meg.

A dolgozatban bemutatott alkalmazás ezekhez képest szűkebb és célzottabb feladatot lát el. Nem teljes oktatási platformként készült, hanem olyan webes alkalmazásként, amely elsősorban matematikai kérdések és feladatok támogatására szolgál. A rendszer célja, hogy a felhasználó ne kizárólag kész választ kapjon, hanem olyan rövid és érthető segítséget, amely a megoldási folyamat megértését támogatja.

A saját alkalmazás egyik fontos jellemzője, hogy a matematikai jellegű kérésekre fókuszál. Ennek megfelelően a rendszernek fel kell ismernie azokat az eseteket is, amikor a felhasználó nem matematikai témájú kérdést tesz fel. Ilyenkor az alkalmazásnak nem általános chatbotként kell működnie, hanem jeleznie kell, hogy csak matematikai kérdések és feladatok támogatására használható. Ez a korlátozás segít abban, hogy a rendszer működése a választott problématerülethez igazodjon.

A rendszer további fontos eleme a beszélgetések mentése és folytathatósága. Ez lehetővé teszi, hogy a felhasználó ne elszigetelt kérdés-válasz formában használja az alkalmazást, hanem korábbi beszélgetéseihez visszatérve folytathassa a tanulási folyamatot. A mappakezelés és az előzmények megjelenítése ezt a használatot szervezettebbé teszi, mivel a felhasználó könnyebben áttekintheti korábbi matematikai kérdéseit és beszélgetéseit.

Összességében a saját alkalmazás a vizsgált megoldásokhoz hasonlóan digitális tanulástámogatást valósít meg, azonban egyszerűbb és célzottabb formában. A hangsúly a matematikai kérdések kezelésén, a segédtanár jellegű válaszadáson, a beszélgetések mentésén és a nem releváns kérések kiszűrésén van. A következő fejezet ennek megfelelően a rendszer követelményeit és működési elvárásait mutatja be.

2. A rendszer követelményei és funkcionális specifikációja

A lehetséges megoldások bemutatása után érdemes meghatározni, hogy a saját alkalmazásnak milyen elvárásoknak kell megfelelnie. Ebben a fejezetben a rendszer célcsoportját, hatókörét, valamint a legfontosabb funkcionális és nem funkcionális követelményeket mutatom be. Ezek alapján válik egyértelművé, hogy az alkalmazás milyen feladatokra készült, milyen működést kell biztosítania és milyen szempontokat kell figyelembe venni a tervezés és a megvalósítás során.

2.1 Célcsoport és felhasználói szerepkörök

A rendszer elsődleges célcsoportját azok a tanulók és hallgatók alkotják, akik matematikai feladatok gyakorlásához vagy megértéséhez szeretnének segítséget kapni. Az alkalmazás különösen olyan helyzetekben lehet hasznos, amikor a felhasználó önállóan tanul és nincs közvetlenül mellette tanár, oktató vagy más segítő személy. Ilyenkor egy webes felületen elérhető, beszélgetésalapú matematikai segéd támogathatja a feladat értelmezését és a megoldási folyamat követését.

Az alkalmazás fő funkciói bejelentkezéshez kötöttek, ezért a rendszerben a központi szerepkör a regisztrált felhasználó. A felhasználó regisztráció után tud bejelentkezni, majd ezt követően érheti el a matematikai tutor funkciókat, a beszélgetések mentését és folytatását, a mappakezelést, valamint a profilhoz kapcsolódó műveleteket.

A regisztrált felhasználó új beszélgetést indíthat, matematikai kérdést vagy feladatot küldhet be, majd a kapott válasz alapján folytathatja a párbeszédet. A rendszer lehetőséget ad arra is, hogy a korábbi beszélgetések később visszakereshetők és megnyithatók legyenek. Ez azért fontos, mert a tanulási folyamat gyakran nem egyetlen kérdésből áll, hanem több egymásra épülő lépésből vagy magyarázatból tevődik össze.

A rendszer nem tartalmaz külön tanári vagy adminisztrátori szerepkört. Ennek oka, hogy a dolgozatban bemutatott alkalmazás személyes használatra készült matematikai tutor prototípus, amelynek célja a felhasználó saját gyakorlásának támogatása. A jogosultságkezelés ezért elsősorban arra irányul, hogy a bejelentkezett felhasználó csak a saját beszélgetéseihez, mappáihoz és profiladataihoz férjen hozzá.

Szerepkör	Leírás	Elérhető funkciók
Regisztrált felhasználó	Bejelentkezett felhasználó, aki a matematikai tutor funkciókat használja	Chat indítás, chat folytatása, mentett beszélgetések megtekintése, mappakezelés, profil megtekintése, jelszó módosítása, fiók törlése

2.1.1. ábra: A rendszer felhasználói szerepköre

2.2 A rendszer célja és hatóköre

A kitűzött cél egy olyan webes alkalmazás elkészítése volt, amely mesterséges intelligencia segítségével támogatja a matematikai feladatok gyakorlását és megértését. Az alkalmazás nem teljes oktatási platformként készült, hanem egy célzott webes rendszerként, amely a matematikai kérdésekre, a beszélgetésalapú segítségnyújtásra és a korábbi beszélgetések folytathatóságára helyezi a hangsúlyt.

A rendszer fő funkciói közé tartozik a felhasználói fiókok kezelése, a matematikai tutor használata, valamint a beszélgetések mentése és rendszerezése. A felhasználó regisztráció és bejelentkezés után új beszélgetést indíthat, matematikai kérdést vagy feladatot küldhet be, majd a kapott válasz alapján folytathatja a párbeszédet. A korábbi beszélgetések később is elérhetők, így a felhasználó visszatérhet egy korábban megkezdett tanulási folyamathoz.

A hatókör része a mappakezelés is, amely lehetővé teszi a beszélgetések egyszerűbb rendszerezését. Ez különösen akkor lehet hasznos, ha a felhasználó több különböző témakörben vagy több alkalommal használja az alkalmazást. A rendszer emellett támogatja a profilhoz kapcsolódó alapvető műveleteket is, például a jelszó módosítását és a fiók törlését.

Az alkalmazás egyik fontos korlátozása, hogy csak matematikai jellegű kérések támogatására készült. Ennek megfelelően a rendszernek fel kell ismernie azokat az eseteket, amikor a felhasználó nem matematikai témájú kérdést tesz fel és ilyenkor elutasító választ kell adnia. Ez azért szükséges, mert a cél nem egy általános chatbot létrehozása, hanem egy szűkebb területre, a matematikai tanulástámogatásra fókuszáló rendszer megvalósítása.

A rendszer hatókörén kívül esik a teljes tananyagkezelés, a tanári felület, az osztálytermi csoportkezelés, a dolgozatok automatikus értékelése és a részletes tanulási analitika. Az alkalmazás nem mobil natív alkalmazásként készült, hanem webes felületen használható. Szintén nem célja az éles üzemi környezethez szükséges teljes

körű biztonsági és skálázhatósági megoldások megvalósítása, mivel a dolgozatban bemutatott rendszer elsődlegesen a matematikai tanulástámogató működés megvalósítására és bemutatására fókuszál.

2.3 Funkcionális követelmények

A funkcionális követelmények azt határozzák meg, hogy a rendszernek milyen konkrét műveleteket kell támogatnia. Ezek alapján írható le, hogy a felhasználó milyen feladatokat tud elvégezni az alkalmazásban és ezekhez a rendszer milyen működést biztosít.

A dolgozatban bemutatott alkalmazás esetében a követelmények elsősorban a felhasználói fiókok kezelésére, a matematikai tutor beszélgetésekre, a beszélgetések mentésére és rendszerezésére, valamint a profilhoz kapcsolódó műveletekre vonatkoznak.

A rendszer működésének központi eleme, hogy a felhasználó bejelentkezés után tudjon matematikai kérdéseket feltenni, azokra AI-alapú választ kapni, majd a beszélgetést később is folytatni.

Emiatt a funkcionális követelmények nemcsak az egyszeri kérdés-válasz működést fedik le, hanem a felhasználóhoz kapcsolódó előzmények, mappák és profiladatok kezelését is. A főbb funkcionális követelmények összefoglalását az *M2.3.1.-es melléklet* tartalmazza.

2.3.1 Felhasználói fiókkezelés

Az alkalmazás használata regisztrációhoz és bejelentkezéshez kötött. Ez azért szükséges, mert a rendszer a felhasználóhoz kapcsolódó adatokat, például beszélgetéseket, mappákat és profiladatokat kezel. A regisztráció során a felhasználó e-mail-cím és jelszó megadásával hozhat létre fiókot, amelyet később a bejelentkezéshez használhat.

A bejelentkezés után a rendszer azonosítani tudja a felhasználót, így biztosítható, hogy mindenki csak a saját beszélgetéseire és adataira férjen hozzá. Ez különösen fontos a mentett beszélgetések és a mappakezelés miatt, mivel ezek személyes tanulási előzményeket tartalmaznak. A fiókkezelés tehát nemcsak belépési funkcióként jelenik meg, hanem a rendszer többi részének alapját is adja.

A hitelesítéshez kapcsolódó működés során a rendszernek kezelnie kell a hibás bejelentkezési adatokat, a hiányzó mezőket, valamint azt az esetet is, amikor a felhasználó védett oldalt próbál elérni bejelentkezés nélkül. Ilyenkor az alkalmazásnak egyértelmű visszajelzést kell adnia, illetve szükség esetén vissza kell irányítania a felhasználót a bejelentkezési felületre.

2.3.2 Matematikai tutor beszélgetések kezelése

A rendszer egyik legfontosabb funkciója a matematikai tutor beszélgetések kezelése. A bejelentkezett felhasználó új beszélgetést indíthat, amelyben matematikai kérdést vagy feladatot küldhet be. A rendszer ezt a kérést továbbítja az AI-alapú tutor modul felé, amely a válasz elkészítésekor a matematikai segítségnyújtásra fókuszál.

A beszélgetés nem egyetlen kérdésre korlátozódik. A felhasználó a kapott válasz alapján további kérdéseket tehet fel, pontosítást kérhet vagy folytathatja a feladat megoldását. Ez a működés közelebb áll egy tutor jellegű tanulási folyamathoz, mint egy egyszerű számológéphez, mert a rendszer nemcsak egy eredményt jelenít meg, hanem lehetőséget ad a párbeszéd folytatására is.

A matematikai válaszadásnál fontos szempont, hogy a rendszer ne általános chatbotként működjön, hanem a tanulási folyamatot támogassa. Ennek megfelelően a válaszok célja a feladat értelmezésének, a következő lépés megtalálásának vagy a felhasználói próbálkozás ellenőrzésének segítése. A matematikai kifejezések megjelenítéséhez a frontend LaTeX/KaTeX alapú megjelenítést használ, így a képletek olvashatóbb formában jelenhetnek meg a felületen.

2.3.3 Beszélgetési előzmények és mappakezelés

A program nemcsak az aktuális beszélgetés kezelésére alkalmas, hanem a korábbi beszélgetések mentését és későbbi megnyitását is támogatja. Ez azért fontos, mert a tanulás gyakran nem egyetlen alkalomhoz kötődik. A felhasználó később visszatérhet egy korábbi feladathoz, újraolvashatja a kapott magyarázatot vagy folytathatja a korábban megkezdett párbeszédet.

A beszélgetési előzmények megjelenítése segít abban, hogy a felhasználó átlátható módon kezelje a korábbi munkáit. Egy matematikai tutor alkalmazásnál ez különösen hasznos lehet, mert a különböző témakörökben feltett kérdések idővel

összegyűlhetnek. A rendszer ezért lehetőséget ad arra is, hogy a felhasználó mappákat hozzon létre és ezekbe rendezze a beszélgetéseit.

A mappakezelés célja nem egy összetett tananyagkezelő rendszer megvalósítása, hanem a beszélgetések egyszerű rendszerezése. A felhasználó így például külön kezelheti az algebrai, függvényekkel kapcsolatos vagy egyéb matematikai témájú beszélgetéseit. Ez a funkció támogatja a tanulási előzmények áttekintését és segíti a felhasználót abban, hogy később könnyebben megtalálja a számára fontos beszélgetéseket.

2.3.4 Profilhoz kapcsolódó műveletek

A profilhoz kapcsolódó funkciók a felhasználói fiók alapvető kezelését biztosítják. A bejelentkezett felhasználó megtekintheti a profiljához tartozó adatokat, valamint elvégezhet bizonyos fiókműveleteket. Ezek közé tartozik a jelszó módosítása és a felhasználói fiók törlése.

A jelszó módosítása azért fontos funkció, mert lehetőséget ad a felhasználónak arra, hogy a fiókja biztonságát saját maga kezelje. A művelet során a rendszernek ellenőriznie kell a megadott adatokat és hibás vagy hiányos kitöltés esetén visszajelzést kell adnia. A felhasználó így érthető módon kap információt arról, ha a módosítás nem hajtható végre.

A fiók törlése szintén fontos profilfunkció, mivel a felhasználó számára biztosítani kell a lehetőséget, hogy megszüntesse a hozzá tartozó fiókot. Ez a művelet érzékenyebb, mint egy egyszerű adatváltoztatás, ezért a felhasználói felületen egyértelműen jelezni kell, hogy a törlés következményekkel járhat. A rendszernek ilyen esetben gondoskodnia kell arról, hogy a felhasználóhoz kapcsolódó adatok kezelése a backend működésének megfelelően történjen.

2.3.5 Nem matematikai kérések kezelése

Az alkalmazás egyik fontos követelménye, hogy a matematikai témájú kérésekre fókuszáljon. Mivel az AI-alapú rendszerek általában sokféle témában képesek választ adni, különösen fontos meghatározni, hogy az adott alkalmazás milyen területen használható. Ebben a rendszerben a cél nem egy általános célú chatbot létrehozása, hanem egy matematikai tutor jellegű segéd megvalósítása.

Ennek megfelelően a rendszernek kezelnie kell azokat az eseteket is, amikor a felhasználó nem matematikai témájú kérést küld be. Ilyenkor az alkalmazásnak nem szabad általános válaszgenerátorként működni, hanem jeleznie kell, hogy csak matematikai kérdések és feladatok támogatására használható. Ez a működés segít abban, hogy az alkalmazás a saját hatókörén belül maradjon.

A nem matematikai kérések elutasítása a felhasználói élmény szempontjából is fontos. A visszajelzésnek rövidnek és érthetőnek kell lennie, hogy a felhasználó egyértelműen lássa, miért nem kapott választ az adott kérdésre. Ez a korlátozás egyben a rendszer értékelhetőségét is javítja, mert világosan elkülöníti a matematikai tutor funkciót az általános chatbot jellegű működéstől.

2.4 Nem funkcionális követelmények

Ezen követelmények nem azt írják le, hogy a rendszer pontosan milyen funkciókat valósít meg, hanem azt, hogy ezeknek a funkcióknak milyen minőségben és milyen működési feltételek mellett kell rendelkezésre állniuk. Egy webalkalmazás esetében ilyen szempont lehet például a használhatóság, a hibakezelés, az alapvető biztonság, a rendszer futtathatósága, valamint az, hogy az alkalmazás korlátai egyértelműen kezelhetők legyenek.

Az alkalmazás egyik fontos nem funkcionális követelménye az egyszerű és átlátható használat. A felhasználónak könnyen el kell érnie a fő funkciókat, például a bejelentkezést, az új beszélgetés indítását, a korábbi beszélgetések megnyitását, a mappák kezelését és a profilhoz kapcsolódó műveleteket. Mivel a rendszer célja a matematikai tanulás támogatása, a felületnek nemcsak technikailag kell működni, hanem abban is segítenie kell, hogy a felhasználó követni tudja a kapott magyarázatokat és a beszélgetés menetét.

A felület jelenlegi formájában elsősorban asztali böngészős használatra készült, ugyanakkor bizonyos elrendezési elemei kisebb kijelzőn is alkalmazkodnak. A főoldali elrendezés mobilon egyszerűbb, egyoszlopos nézetre válthat, nagyobb kijelzőn pedig kétoszlopos szerkezetben jelenik meg. Ez azt jelenti, hogy a rendszer alapvető szinten kisebb kijelzőn is használható, de nem tekinthető teljesen mobilra optimalizált alkalmazásnak.

A rendszer nem tartalmaz külön mobilra tervezett navigációt vagy részletesen finomhangolt mobilos felületet. Egyes elemek, például a chatlista, az üzenetpanel vagy

a navigáció kisebb kijelzőn szűkösebbé válhatnak. Emiatt a rendszer jelenlegi állapotában inkább asztali használatra optimalizált célzott webes alkalmazásként értelmezhető, míg a teljes mobilos optimalizálás a továbbfejlesztési lehetőségek közé tartozik.

A rendszernek kezelnie kell a hibás vagy sikertelen műveleteket is. Ilyen lehet például a hibás bejelentkezési adat, a hiányzó mező, a sikertelen hálózati kérés vagy az, ha az AI-modell nem érhető el. Ezekben az esetekben fontos, hogy az alkalmazás ne váratlanul álljon le, hanem érthető visszajelzést adjon a felhasználónak. A hibaüzenetek célja, hogy a felhasználó számára egyértelmű legyen, mi történt és szükség esetén milyen lépést tehet.

Biztonsági szempontból lényeges, hogy a felhasználóhoz tartozó adatok csak bejelentkezés után legyenek elérhetők. A rendszernek biztosítania kell, hogy a felhasználó csak a saját beszélgetéseire, mappáira és profiladataira férjen hozzá. A dolgozatban bemutatott alkalmazás prototípus jellegű, ezért nem célja egy éles üzemi környezet teljes biztonsági követelményeinek megvalósítása, de az alapvető hitelesítési és jogosultságkezelési működésnek jelen kell lennie.

A reprodukálható futtatás szintén fontos szempont, mivel a rendszernek nemcsak a fejlesztő saját környezetében, hanem ellenőrizhető módon más környezetben is elindíthatónak kell lennie. Ennek támogatására a projekt Docker Compose alapú futtatást használ, amely megkönnyíti az adatbázis és a backend indítását. A környezeti változók használata lehetővé teszi, hogy a szükséges beállítások, például a titkos kulcsok és a modellhez kapcsolódó tokenek külön konfigurálhatók legyenek.

Az AI-alapú működés miatt külön figyelmet kell fordítani a válaszok korlátaira is. A rendszer által adott válaszok támogathatják a tanulási folyamatot, de nem tekinthetők minden esetben hibátlannak. Emiatt fontos követelmény, hogy a válaszok mintafeladatokon keresztül ellenőrizhetők legyenek. A rendszernek emellett kezelnie kell azt az esetet is, ha a modell nem érhető el vagy nem tud megfelelő választ generálni. A rendszer főbb nem funkcionális követelményeinek összefoglalását az *M2.4.1-es melléklet* tartalmazza.

2.5 Felhasználói folyamatok

A követelmények meghatározása után érdemes bemutatni, hogy a felhasználó milyen lépéseken keresztül használja az alkalmazást. A felhasználói folyamatok segítenek

megérteni, hogyan kapcsolódnak egymáshoz a korábban bemutatott funkciók és milyen útvonalakon keresztül érhetők el a rendszer fő szolgáltatásai.

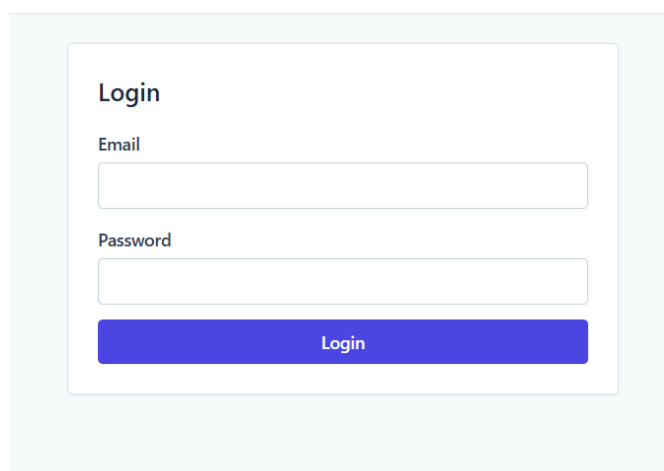
2.5.1 Regisztráció és bejelentkezés folyamata

Az alkalmazás használatának első lépése a felhasználói fiók létrehozása és/vagy a bejelentkezés. Mivel a rendszer mentett beszélgetéseket, mappákat és profilhoz kapcsolódó adatokat kezel, a fő funkciók csak azonosított felhasználó számára érhetők el. Ez biztosítja, hogy a felhasználó később is visszatérhessen a saját beszélgetéseihez és azokat ne más felhasználó adataival keverje össze a rendszer.

A regisztráció során a felhasználó e-mail címet és jelszót ad meg. A sikeres regisztráció után a létrehozott fiókkal be tud jelentkezni az alkalmazásba. A bejelentkezési folyamat célja, hogy a rendszer ellenőrizze a megadott adatokat, majd sikeres hitelesítés esetén hozzáférést adjon a matematikai tutor munkaterülethez. A bejelentkezés után válik elérhetővé az új beszélgetés indítása, a korábbi beszélgetések megnyitása, a mappakezelés és a profiloldal.

A folyamat során fontos szerepet kap a hibakezelés is. A rendszernek kezelnie kell például az üresen hagyott mezőket, a hibás bejelentkezési adatokat, illetve azt az esetet, amikor a regisztráció során a megadott e-mail cím már nem használható. Ilyenkor a felhasználónak egyértelmű visszajelzést kell kapnia, hogy javítani tudja a megadott adatokat.

A bejelentkezési és regisztrációs felület egyszerű felépítésű, mivel ezeknél a képernyőknél a cél nem az összetett navigáció, hanem az, hogy a felhasználó gyorsan el tudja végezni a belépéshez szükséges műveleteket.

A screenshot of a login form. The form is titled "Login" and is contained within a light blue rectangular frame. It features two input fields: "Email" and "Password". Below these fields is a prominent blue button with the text "Login" in white. The form is set against a light gray background.

2.5.1.1. ábra: Login felület

A sikeres bejelentkezést követően a felhasználó a fő munkaterületre jut, ahol elindíthatja az első matematikai tutor beszélgetést. Ezzel a regisztrációs és bejelentkezési folyamat lezárul és megkezdődhet az alkalmazás tényleges tanulástámogató használata.

2.5.2 Beszélgetés indítása és folytatása

A sikeres bejelentkezés után a felhasználó a fő munkaterületre jut, ahol elindíthatja a matematikai tutorral folytatott beszélgetést. Ez a felület az alkalmazás központi része, mivel itt történik a matematikai kérdések beküldése, a válaszok megjelenítése, valamint a korábbi beszélgetések kezelése is.

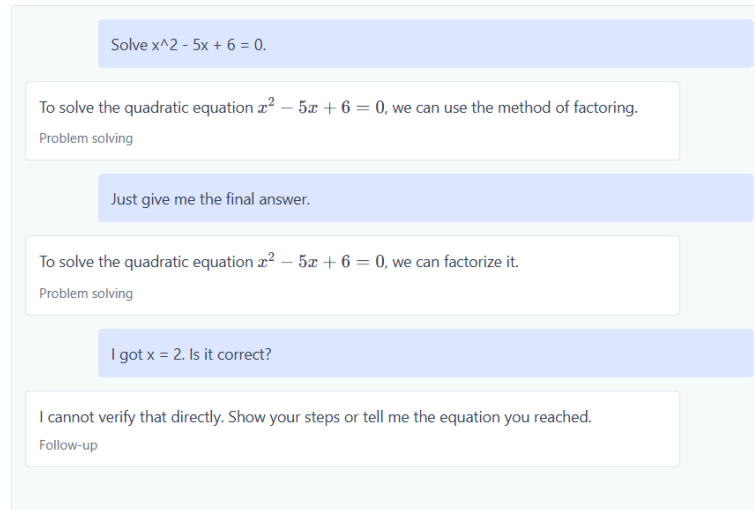
Új beszélgetés indításakor a felhasználó megad egy matematikai kérdést vagy feladatot. A rendszer ezt a kérést továbbítja az AI-alapú tutor modul felé, majd a válasz megjelenik a chatfelületen. A cél nem egyszerűen az, hogy a felhasználó egy kész választ kapjon, hanem az, hogy a rendszer a tanulási folyamatot támogató segítséget adjon. Ez lehet például egy következő lépés, egy rövid magyarázat vagy egy olyan iránymutatás, amely segíti a feladat megértését.

A beszélgetés folytatható, tehát a felhasználó a kapott válasz után további kérdéseket tehet fel ugyanabban a témában. Ez azért fontos, mert egy matematikai feladat megértése gyakran nem egyetlen kérdésből és válaszból áll. Előfordulhat, hogy a felhasználó pontosítást kér, ellenőriztetni szeretné a saját próbálkozását vagy segítséget kér a következő lépéshez. A párbeszédalapú működés emiatt jobban illeszkedik a tutor jellegű tanulási folyamathoz, mint egy egyszeri válaszgenerálás.

A rendszer a beszélgetéseket a felhasználóhoz kapcsolva tárolja, így a korábbi kérdések és válaszok később is visszakereshetők. Ez lehetővé teszi, hogy a felhasználó ne veszítse el a korábban megkezdett tanulási folyamatot, hanem később visszatérhessen hozzá. A fő chatfelület ezért nemcsak az aktuális kérdés beküldésére szolgál, hanem a tanulási előzmények kezelésének egyik alapvető felülete is.

Math Tutor Chat

English-only, mathematics-only tutoring. Ask for a new task, help with solving, or follow-up hints.



2.5.2.1. ábra: Beszélgetés indítása az AI ttorral

2.5.3 Mentett beszélgetések és mappák kezelése

A matematikai tutor használata során fontos szempont, hogy a felhasználó ne veszítse el a korábban megkezdett beszélgetéseit. Egy tanulási folyamat gyakran több alkalomból áll, ezért hasznos, ha a rendszer eltárolja a korábbi kérdéseket és válaszokat. Így a felhasználó később visszatérhet egy már megkezdett feladathoz, újraolvashatja a korábbi magyarázatokat vagy folytathatja az adott témához kapcsolódó párbeszédet.

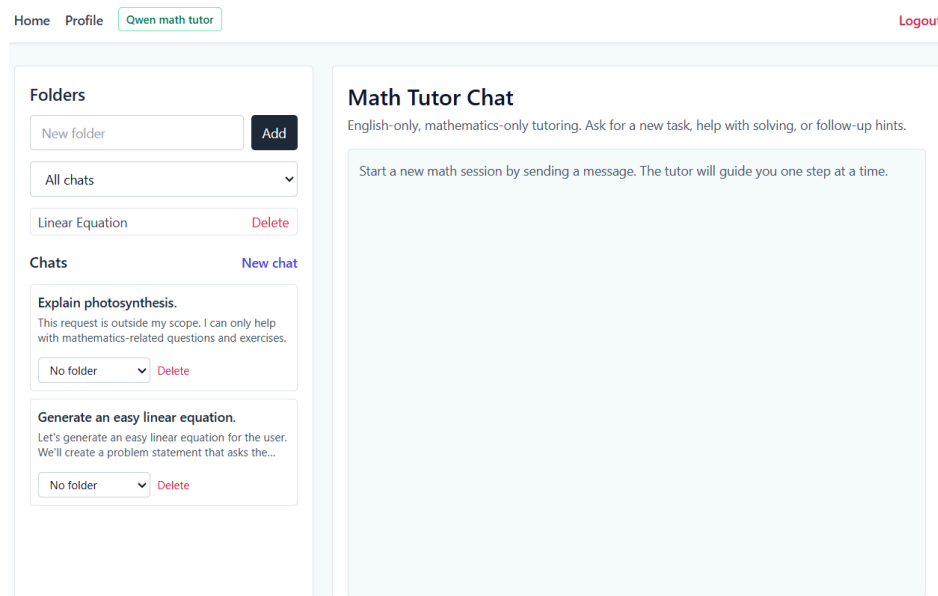
A fő munkaterületen a felhasználó megtekintheti a mentett beszélgetéseit és kiválaszthatja azt, amelyikkel tovább szeretne dolgozni. Egy korábbi beszélgetés megnyitásakor a rendszer betölti az adott beszélgetéshez tartozó üzeneteket, így a felhasználó látja a korábbi kérdéseit és a tutor válaszait. Ez segíti a tanulási előzmények követését, különösen akkor, ha a felhasználó több különböző matematikai témával foglalkozik.

A beszélgetések rendszerezését a mappakezelés támogatja. A felhasználó mappákat hozhat létre, majd ezekbe rendezheti a hozzájuk kapcsolódó beszélgetéseket. Ez egyszerűbbé teszi a korábbi munkák áttekintését, mert a felhasználó például külön kezelheti az algebrai, függvényekkel kapcsolatos vagy más témájú beszélgetéseit. A mappák használata nem teljes tananyagkezelő rendszerként jelenik meg, hanem a mentett beszélgetések egyszerű csoportosítását szolgálja.

A rendszer lehetőséget ad a beszélgetések törlésére is. Ez akkor lehet hasznos, ha a felhasználó már nem szeretne megtartani egy korábbi párbeszédet vagy rendezni

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

szeretné az előzményeit. A mappák törlésekor figyelembe kell venni, hogy a hozzájuk tartozó beszélgetések kezelése a backend működésének megfelelően történik, ezért ezt a műveletet a felhasználói felületen egyértelműen kell jelezni.



2.5.3.1. ábra: Mentett beszélgetések és mappák megjelenítése a főfelületen

2.5.4 Profilkezelés és profilműveletek

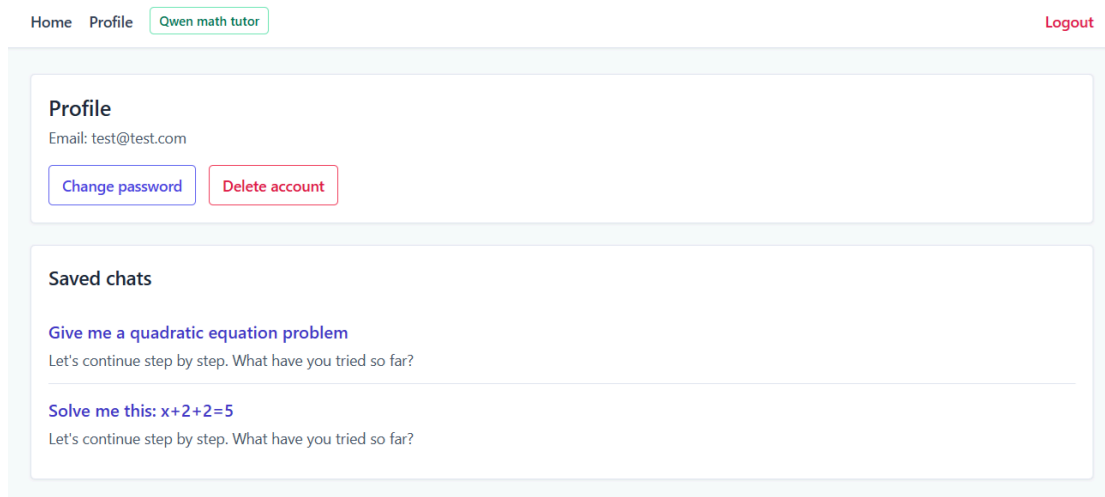
A bejelentkezett felhasználó az alkalmazásban nemcsak matematikai beszélgetéseket indíthat, hanem a saját fiókjához kapcsolódó alapvető műveleteket is kezelheti. Ezek a funkciók a profiloldalon érhetők el, ahol a felhasználó megtekintheti a fiókjához tartozó adatokat, valamint elvégezhet bizonyos módosításokat.

A profilkezelés egyik fontos része a jelszó módosítása. Erre azért van szükség, mert a felhasználónak lehetőséget kell biztosítani arra, hogy a fiókja hozzáférési adatait később is kezelni tudja. A jelszóváltoztatás során a rendszernek ellenőriznie kell a megadott adatokat és hibás vagy hiányos kitöltés esetén egyértelmű visszajelzést kell adnia. Ez segíti a felhasználót abban, hogy javítani tudja a hibás adatokat és a műveletet sikeresen végrehajtsa.

A profiloldalon elérhető másik fontos művelet a felhasználói fiók törlése. Ez érzékenyebb művelet, mint egy egyszerű adatlekérdezés vagy jelszóváltoztatás, ezért a felhasználói felületen egyértelműen jelezni kell, hogy a törlés következményekkel jár. A fiók törlése után a felhasználó már nem fér hozzá a korábban hozzá tartozó adatokhoz, ezért ezt a műveletet csak megerősítés után érdemes végrehajtani.

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

A profilhoz kapcsolódó műveletek szerepe az, hogy a felhasználó önállóan tudja kezelni a saját fiókját. Ez illeszkedik az alkalmazás személyes használatra épülő működéséhez, ahol a beszélgetések, mappák és profiladatok egy adott bejelentkezett felhasználóhoz tartoznak.



2.5.4.1. ábra: A profiloldal és a fiókhoz kapcsolódó műveletek

3. Felhasznált technológiák

A dolgozatban bemutatott alkalmazás több, egymást kiegészítő technológiára épül. A rendszer webes felülete frontend oldalon készült el, a szerveroldali működést backend alkalmazás biztosítja, az adatok tárolása adatbázisban történik, míg a matematikai tutor funkciót egy mesterséges intelligencia alapú nyelvi modell támogatja. A technológiák kiválasztásánál fontos szempont volt, hogy a rendszer webes környezetben használható legyen, kezelni tudja a felhasználói fiókokat és beszélgetéseket, valamint lehetőséget adjon az AI-alapú válaszadás beépítésére.

A frontend oldalon a Next.js^[2], a TypeScript^[3] és a Tailwind CSS^[17] kapott szerepet, míg a backend FastAPI^[4] alapú. Az adatok tárolását PostgreSQL^[5] adatbázis biztosítja, a lokális futtatást pedig Docker^[6] segíti. Az AI tutor modul a Qwen/Qwen2.5-Math-1.5B-Instruct^[18] modellre épül, amely Hugging Face Transformers^[19] környezetben futtatható.

3.1 Frontend technológiák

A felhasználói felület Next.js^[2] használatával készült. A Next.js^[2] egy React^[20] alapú keretrendszer, amely alkalmas modern webalkalmazások fejlesztésére. A projektben a frontend feladata a felhasználói interakciók kezelése, a bejelentkezési és regisztrációs felületek megjelenítése, a fő chat munkaterület biztosítása, valamint a profilhoz és mappakezeléshez kapcsolódó képernyők megvalósítása.

A frontend TypeScript^[3] nyelven készült, amely a JavaScript^[21] típusosabb változataként használható. A TypeScript^[3] előnye, hogy a fejlesztés során segít a hibák korábbi felismerésében, mivel a változók, függvények és komponensek típusai pontosabban meghatározhatók. Ez különösen hasznos egy olyan alkalmazásnál, ahol többféle adatot kell kezelni, például felhasználói adatokat, chatüzeneteket, mappákat és API-válaszokat.

A felület megjelenítéséhez Tailwind CSS^[17] került felhasználásra. Ez lehetővé teszi, hogy a stílusok közvetlenül a komponensekhez kapcsolódva, előre definiált segédosztályokkal legyenek megadva. A Tailwind CSS használata segít az egységes megjelenés kialakításában, például a gombok, űrlapok, kártyák, oldalsávok és elrendezések formázásánál.

A projektben szerepet kap a react-hot-toast^[22] könyvtár is, amely rövid visszajelzések megjelenítésére használható. Ilyen visszajelzés lehet például egy sikeres művelet, egy hibás bejelentkezés vagy egy sikertelen kérés jelzése. A matematikai kifejezések megjelenítéséhez KaTeX^[23] került alkalmazásra, amely lehetővé teszi, hogy a képletek olvashatóbb, matematikai formában jelenjenek meg a chatfelületen.

A frontend tehát nemcsak a felület megjelenítéséért felel, hanem a felhasználói folyamatok irányításáért is. Ide tartozik a bejelentkezés, a regisztráció, az új beszélgetés indítása, a korábbi beszélgetések megnyitása, a mappák kezelése, valamint a profiloldalhoz kapcsolódó műveletek elérése.

3.2 Backend technológiák

A szerveroldali működés FastAPI^[4] keretrendszerrel készült. A FastAPI^[4] egy Python^[8] alapú webes keretrendszer, amely REST API-k^[24] fejlesztésére használható. A projektben a backend feladata a frontendtől érkező kérések feldolgozása, a felhasználói műveletek kezelése, az adatbázissal való kommunikáció, valamint az AI tutor modul meghívása.

A backend egyik legfontosabb szerepe a felhasználói fiókok kezelése. Ide tartozik a regisztráció, a bejelentkezés, az aktuális felhasználó adatainak lekérése, a jelszó módosítása és a fiók törlése. A rendszer a bejelentkezés után JWT-alapú^[25] hitelesítést használ, amely lehetővé teszi, hogy a védett API-végpontokat csak azonosított felhasználók érhék el. Ez azért fontos, mert a beszélgetések, mappák és profiladatok mindig egy adott felhasználóhoz kapcsolódnak.

A backend kezeli a matematikai tutor beszélgetések életciklusát is. Új beszélgetés indításakor a rendszer létrehoz egy chatet, eltárolja a felhasználó első üzenetét, majd meghívja az AI tutor modult. A tutor válasza ezután szintén mentésre kerül, így a beszélgetés később is visszakereshető. Meglévő beszélgetés esetén a felhasználó újabb üzenetet küldhet, amelyet a rendszer az adott chathez kapcsolva kezel.

A backend REST API-végpontokon^[24] keresztül kommunikál a frontenddel. Az autentikációhoz külön végpontok tartoznak, például a regisztráció, a bejelentkezés, a profiladatok lekérése, a jelszó módosítása és a fiók törlése. A chatfunkciókhoz tartozó végpontok biztosítják az új beszélgetés indítását, a folytató üzenetek küldését, a beszélgetések listázását, egy konkrét beszélgetés megnyitását és a beszélgetések

törlését. A mappakezeléshez szintén külön végpontok tartoznak, amelyek a mappák létrehozását, listázását és törlését teszik lehetővé.

Az adatbázissal való kapcsolat kezeléséhez SQLAlchemy ORM^[26] került felhasználásra. Az ORM lehetővé teszi, hogy az adatbázis táblái Python osztályokként jelenjenek meg a kódban. Ez áttekinthetőbbé teszi a felhasználók, beszélgetések, üzenetek és mappák kezelését, mivel az adatbázis-műveletek nem közvetlen SQL-lekérdezések formájában jelennek meg a kódban, hanem objektumokon keresztül történnek.

A backend hibakezelési feladatokat is ellát. Ilyen eset lehet például a hibás bejelentkezési adat, a hiányzó vagy érvénytelen token, a nem létező beszélgetés vagy mappa, illetve a hibás bemeneti adat. Ezekben az esetekben a rendszer megfelelő HTTP-választ ad vissza, amelyet a frontend felhasználói visszajelzésként tud megjeleníteni.

A backend tehát központi szerepet tölt be az alkalmazás működésében. Összeköti a felhasználói felületet az adatbázissal és az AI tutor modullal, miközben kezeli a hitelesítést, az adatok mentését, a chatfolyamatokat, a mappaműveleteket és a hibaágakat.

Terület	Backend szerepe
<i>Hitelesítés</i>	Regisztráció, bejelentkezés és JWT-kezelés
<i>Felhasználói adatok</i>	Profiladatok lekérése, jelszó módosítása, fiók törlése
<i>Chatkezelés</i>	Új beszélgetés indítása, folytatható üzenetek kezelése, beszélgetések listázása
<i>Mappakezelés</i>	Mappák létrehozása, törlése és beszélgetések mappába rendezése
<i>Adatbázis-kapcsolat</i>	Felhasználók, beszélgetések, üzenetek és mappák mentése PostgreSQL-be
<i>AI-integráció</i>	A Qwen-alapú tutor modul meghívása és válaszának visszaadása
<i>Hibakezelés</i>	Érvénytelen tokenek, hibás bemenetek és nem létező erőforrások kezelése

3.2.1. ábra: A backend fő feladatai

3.3 Adatbázis és konténerizáció

Az alkalmazás adatait PostgreSQL^[5] adatbázis tárolja. A PostgreSQL^[5] egy relációs adatbázis-kezelő rendszer, amely alkalmas strukturált adatok, például felhasználói fiókok, beszélgetések, üzenetek és mappák tárolására. A projektben azért volt megfelelő választás, mert a rendszer adatai jól modellezhetők egymással kapcsolatban álló

táblákkal. Egy felhasználóhoz több beszélgetés és több mappa tartozhat, a beszélgetések pedig több üzenetből épülnek fel.

Az adatbázisban a fő entitások a felhasználók, a beszélgetések, a beszélgetési üzenetek és a mappák. A felhasználóhoz kapcsolódnak a saját beszélgetései és mappái, így biztosítható, hogy minden bejelentkezett felhasználó csak a saját adatait érje el. A beszélgetésekhez külön üzenetek tartoznak, amelyek a felhasználói és az asszisztensi válaszokat is eltárolják. A mappák célja a beszélgetések egyszerű rendszerezése.

Az adatbázissal való kapcsolat kezeléséhez SQLAlchemy ORM^[26] került felhasználásra. Az ORM lényege, hogy az adatbázis táblái és kapcsolatai Python osztályokon keresztül kezelhetők. Ez áttekinthetőbbé teszi a fejlesztést, mert a rendszerben szereplő objektumok, például a *User*, *Chat*, *ChatMessage* és *Folder* modellek közvetlenül kapcsolódnak az adatbázis szerkezetéhez.

A projekt jelenlegi állapotában az adatbázistáblák a backend indulásakor jönnek létre. Ez egyszerűvé teszi az alkalmazás jelenlegi változatának futtatását és kipróbálását, mert nincs szükség külön adatbázis-migrációs lépésekre. Ugyanakkor ez korlátot is jelent, mert egy élesebb, hosszabb távon karbantartott rendszerben célszerű lenne külön migrációs eszközt használni, például az adatbázisséma változásainak biztonságos kezelésére.

A lokális futtatást Docker^[6] segíti. A Docker^[6] lehetővé teszi, hogy a projekt több szolgáltatása, például az adatbázis és a backend, egy közös konfiguráció alapján induljon el. Ez szakdolgozati szempontból is hasznos, mert a rendszer nemcsak a fejlesztő saját gépén, hanem dokumentált lépések alapján más környezetben is ellenőrizhetővé válik.

A projekt környezeti változókat használ a futtatási beállítások kezelésére. Ilyen beállítás például az adatbázis kapcsolati útvonala, a JWT^[25] működéséhez szükséges titkos kulcs, valamint a Qwen modellhez^[18] kapcsolódó Hugging Face token. Ez azért előnyös, mert az érzékeny vagy környezetfüggő adatok nem közvetlenül a forráskódban szerepelnek, hanem külön konfigurációs fájlban vagy környezeti változóként adhatók meg.

A konténerizált futtatás és a környezeti változók használata együtt hozzájárul ahhoz, hogy a rendszer reprodukálhatóbb legyen. A bíráló vagy fejlesztő a dokumentált lépések alapján el tudja indítani az adatbázist és a backendet, majd külön el tudja indítani a frontend alkalmazást. Ez megkönnyíti a projekt ellenőrzését, tesztelését és bemutatását.

Technológia	Szerepe a projektben
<i>PostgreSQL</i>	Felhasználók, beszélgetések, üzenetek és mappák tárolása
<i>SQLAlchemy ORM</i>	Adatbázismodellek és kapcsolatok kezelése Python osztályokon keresztül
<i>Docker</i>	Adatbázis és backend lokális indítása dokumentált konfiguráció alapján
<i>Környezeti változók</i>	Adatbázis-kapcsolat, titkos kulcsok és modellbeállítások kezelése

3.3.1.ábra: Az adatkezeléshez és futtatáshoz kapcsolódó technológiák

3.4 AI modell és Python könyvtárak

Az alkalmazás egyik központi eleme az AI-alapú matematikai tutor modul. Ennek feladata, hogy a felhasználó matematikai jellegű kérdéseire vagy feladataira tanulást támogató választ adjon. A rendszer nem általános chatbotként használja a nyelvi modellt, hanem matematikai tutor szerepkörben. Ennek megfelelően a cél nem feltétlenül a teljes megoldás azonnali közlése, hanem a következő lépés, egy rövid magyarázat vagy egy iránymutatás megadása.

A tutor funkció a Qwen/Qwen2.5-Math-1.5B-Instruct^[18] modellre épül. Ez a modell a Qwen modelleszalád^[7] matematikai feladatokra specializált változatai közé tartozik, ezért alkalmas lehet matematikai jellegű kérdések feldolgozására. A projektben a modell használata környezeti változókon keresztül konfigurálható. Ilyen beállítás például a modell azonosítója, a generálható tokenek maximális száma és a válaszadás hőmérsékleti paramétere.

A modell betöltése és futtatása a Hugging Face Transformers^[19] könyvtár segítségével történik. A Transformers lehetővé teszi előre betanított nyelvi modellek betöltését és használatát Python környezetben. A modell működéséhez Hugging Face token szükséges, amelyet a rendszer környezeti változóként kezel. Ez azért előnyös, mert az érzékeny hozzáférési adat nem kerül közvetlenül a forráskódba.

A modell futtatásának mélytanulási háttérét a PyTorch^[27] biztosítja. A futtatási környezet teljesítménye jelentősen befolyásolja az AI tutor válaszüdejét. Amennyiben elérhető GPU, a rendszer 4 bites kvantálást is használhat, amelyet a bitsandbytes^[28] könyvtár támogat. Ez csökkentheti a modell futtatásához szükséges erőforrásokat. Ha GPU nem áll rendelkezésre, a modell CPU-n is futtatható, de ebben az esetben a

működés lassabb és memóriaigényesebb lehet. Ez az alkalmazás egyik fontos technikai korlátja.

Az AI tutor működését rendszerutasítás is szabályozza. Ennek célja, hogy a modell matematikai témára fókuszáljon, ne általános asszisztensként válaszoljon és a nem matematikai jellegű kéréseket elutasítsa. A rendszerutasítás emellett azt is meghatározza, hogy a válaszok tutor jellegűek legyenek vagyis lehetőség szerint ne azonnal teljes megoldást adjanak, hanem a következő lépésre vagy egy rövid iránymutatásra koncentráljanak.

A rendszer a válaszokat különböző kategóriák szerint kezeli. Ilyen kategória lehet például a feladatgenerálás, a problémamegoldás, a folytató kérdés vagy a nem matematikai kérdés. Feladatgenerálás esetén a cél egyetlen matematikai feladat létrehozása, megoldás nélkül. Problémamegoldás és folytató kérdés esetén a rendszer rövid, tanulást támogató választ ad. Nem matematikai kérdés esetén a tutor elutasító választ küld, jelezve, hogy az alkalmazás csak matematikai kérdések és feladatok támogatására használható.

Az AI-alapú működés egyik fontos korlátja, hogy a modell válaszai nem tekinthetők minden esetben hibátlanoknak. Előfordulhat pontatlan, hiányos vagy félrevezető válasz, ezért a rendszer használatánál szükséges a válaszok ellenőrzése. A modell tehát tanulástámogató segédeszközként jelenik meg, nem pedig olyan megoldásként, amely minden esetben garantáltan helyes matematikai választ ad.

A rendszernek kezelnie kell azokat az eseteket is, amikor a modell nem érhető el. Ilyen lehet például a hiányzó Hugging Face token, a sikertelen modellbetöltés vagy az erőforráshiány. Ilyen esetben a felhasználó számára jelezni kell, hogy az AI tutor nem megfelelően konfigurált vagy átmenetileg nem elérhető. Ez azért fontos, mert a rendszer többi része, például a bejelentkezés, a beszélgetések mentése és a mappakezelés ettől még ellenőrizhető maradhat.

Az AI tutor modul és a backend működéséhez több Python könyvtár kapcsolódik. A modell betöltéséhez és futtatásához a Transformers, a PyTorch^[27], a huggingface_hub és a bitsandbytes^[28] kap szerepet. A backend működéséhez a FastAPI^[4], az Uvicorn, a Pydantic^[29] és az SQLAlchemy^[26] szükséges. A hitelesítéshez és felhasználókezeléshez kapcsolódóan a passlib, a bcrypt és a python-jose könyvtárak is fontosak. A projektben használt fontosabb Python könyvtárak összefoglalása az *M3.4.1-es mellékletben* található.

3.5 Fejlesztői környezet és verziókezelés

A projekt fejlesztése Visual Studio Code^[1] környezetben történt. Ez a fejlesztői környezet alkalmas volt a több részből álló alkalmazás kezelésére, mivel egy helyen volt szerkeszthető a frontend, a backend, a konfigurációs fájlok és a dokumentáció. A projekt felépítése külön mappákba szervezi a kliensoldali és szerveroldali részeket, ami átláthatóbbá teszi a fejlesztést.

A verziókezelés Git használatával történt, a forráskód pedig GitHub^[30] repóban érhető el. Ez lehetővé teszi a módosítások követését, a korábbi állapotok visszakeresését, valamint azt, hogy a projekt a konzulens vagy a bíráló számára is ellenőrizhető legyen. A GitHubon elérhető repó nemcsak a forráskódot, hanem a futtatáshoz és értékeléshez szükséges dokumentációkat is tartalmazza.

A projektben több Markdown alapú dokumentációs fájl is készült. Ezek között megtalálható az architektúra, az API-végpontok, az adatmodell, a reprodukálható futtatás, a tesztelés és az AI tutor működésének leírása. Ezek a dokumentumok segítik, hogy az alkalmazás ne csak működő prototípusként, hanem átlátható mérnöki munkaként is értékelhető legyen.

A fejlesztői környezet, a verziókezelés és a dokumentáció együttesen támogatják a projekt karbantarthatóságát és ellenőrizhetőségét. A következő fejezetben a rendszer felépítése és fő komponensei kerülnek bemutatásra.

4. Rendszerterv

Egy webalkalmazás tervezésénél fontos, hogy a különböző komponensek feladatai világosan elkülönüljenek egymástól. A dolgozatban bemutatott rendszer is ilyen felépítést követ. A felhasználói felület, a szerveroldali logika, az adatbázis és az AI-alapú tutor modul külön szerepet kapnak, de egymással együttműködve biztosítják az alkalmazás működését. Ez a felépítés átláthatóbbá teszi a rendszer működését és megkönnyíti az egyes részek későbbi módosítását vagy továbbfejlesztését.

4.1 A rendszer architektúrája

Az alkalmazás több, egymástól elkülönülő, de együttműködő komponensből épül fel. A felhasználó a webes felületen keresztül lép kapcsolatba a rendszerrel, a felületen végzett műveletek pedig API-kéréseken keresztül jutnak el a backendhez. A backend felel a kérések feldolgozásáért, a jogosultságok ellenőrzéséért, az adatok mentéséért, valamint az AI tutor modul meghívásáért.

A rendszer működésének központi eleme a backend, mivel ez kapcsolja össze a felhasználói felületet, az adatbázist és az AI modult. A frontend önmagában nem fér hozzá közvetlenül az adatbázishoz, hanem minden műveletet a backend végpontjain keresztül végez. Ez a felépítés segít abban, hogy az adatkezelés és a jogosultságellenőrzés egy központi rétegben történjen.

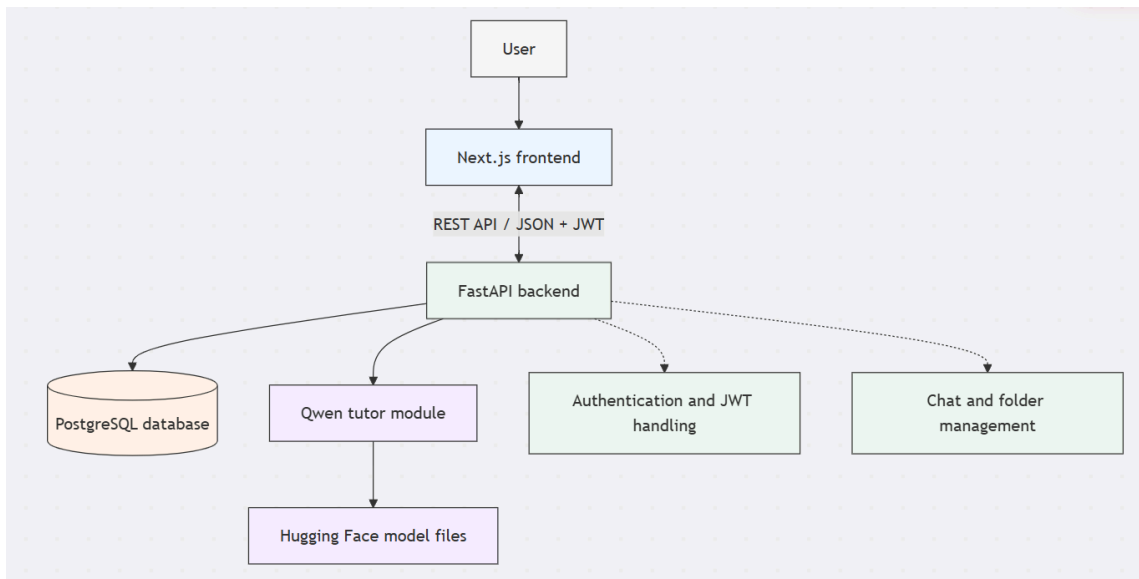
Egy tipikus használati folyamat során a felhasználó bejelentkezik, majd a fő felületen matematikai kérdést küld be. A frontend ezt a kérést továbbítja a backendnek, amely ellenőrzi a felhasználó hitelesítését, létrehozza vagy frissíti az adott beszélgetést, majd továbbítja a kérdést az AI tutor modul felé. A tutor által generált válasz visszakerül a backendhez, amely eltárolja azt az adatbázisban, majd elküldi a frontendnek megjelenítésre.

Az adatbázis a rendszer tartós adattárolási rétegeként működik. Itt tárolódnak a felhasználók, a beszélgetések, a beszélgetésekhez tartozó üzenetek és a mappák. Ennek köszönhetően a felhasználó később is visszatérhet a korábbi beszélgetéseihez, illetve folytathatja a már megkezdett tanulási folyamatot.

Az AI tutor modul nem önálló felhasználói felületként működik, hanem a backend által meghívott komponensként. Ez azt jelenti, hogy a felhasználó közvetlenül

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

nem a modellel kommunikál, hanem az alkalmazáson keresztül küld kérdést, a modell válasza pedig a rendszer saját szabályai és adatkezelési folyamatai szerint jelenik meg a chatfelületen.



4.1.1. ábra: A rendszer fő komponensei és kapcsolataik

Az ábra alapján látható, hogy a frontend és a backend között REST-alapú kommunikáció történik. A backend két irányba kapcsolódik tovább, egyrészt az adatbázishoz, ahol a felhasználói és beszélgetési adatok tárolódnak, másrészt az AI tutor modulhoz, amely a matematikai válaszok előállításáért felel. Ez a felépítés átláthatóvá teszi, hogy az egyes komponensek milyen szerepet töltenek be a teljes rendszer működésében.

4.2 Frontend, backend és adatbázis kapcsolata

A rendszer működésében a frontend, a backend és az adatbázis egymásra épülő szerepet tölt be. A felhasználó minden műveletet a frontend felületen keresztül indít el, például bejelentkezik, új beszélgetést kezd, üzenetet küld, mappát hoz létre vagy megnyit egy korábbi beszélgetést. Ezek a műveletek azonban nem közvetlenül az adatbázison történnek, hanem a frontend API-kéréseket küld a backend felé.

A frontend feladata elsősorban az adatok megjelenítése és a felhasználói interakciók kezelése. Amikor a felhasználó például beír egy matematikai kérdést és elküldi azt, a frontend összeállítja a megfelelő kérést, majd továbbítja a backend egyik API-végpontjára. A kérés tartalmazza a szükséges adatokat, például az üzenet szövegét, valamint hitelesített művelet esetén a felhasználóhoz tartozó token is.

A backend a beérkező kérések feldolgozásáért felel. Először ellenőrzi, hogy a kéréshez tartozik-e érvényes hitelesítés, majd megvizsgálja, hogy a kért művelet végrehajtható-e. Például egy beszélgetés megnyitásakor ellenőriznie kell, hogy az adott beszélgetés létezik-e és valóban az aktuális felhasználóhoz tartozik-e. Ez biztosítja, hogy a felhasználók csak a saját adataikhoz férjenek hozzá.

Az adatbázis a rendszer tartós adattárolási rétegeként működik. Itt tárolódnak a felhasználói fiókok, a beszélgetések, az üzenetek és a mappák. Új beszélgetés indításakor a backend létrehozza a megfelelő adatbázisrekordokat, eltárolja a felhasználó üzenetét, majd az AI tutor választ is ugyanahhoz a beszélgetéshez kapcsolja. Ennek köszönhetően a beszélgetés később újra megnyitható és folytatható.

A backend és az adatbázis kapcsolata azért fontos, mert a rendszer több olyan funkciót is biztosít, amely felhasználóhoz kötött adatokra épül. Ilyen például a mentett beszélgetések listázása, egy korábbi beszélgetés részleteinek megjelenítése, a mappák kezelése vagy a profilhoz kapcsolódó műveletek. Ezeknél a backend mindig az adatbázisból olvassa ki a szükséges információkat, majd strukturált válaszként továbbítja azokat a frontend számára.

A frontend a backend válasza alapján frissíti a felületet. Ha például egy új üzenet elküldése sikeres, akkor a chatfelületen megjelenik a felhasználó üzenete és az asszisztens válasza. Ha hiba történik, például a felhasználó nincs bejelentkezve, nem létező beszélgetést próbál megnyitni vagy hibás adatot küld be, akkor a backend hibaválaszt ad, amelyet a frontend felhasználói visszajelzésként jeleníthet meg.

A kapcsolat tehát nem egyszerű adatküldésből áll, hanem egy több lépésből felépülő folyamatból. A frontend kezeli a felhasználói műveleteket, a backend ellenőrzi és feldolgozza a kéréseket, az adatbázis pedig biztosítja az adatok tartós tárolását. Ez az elkülönítés átláthatóbbá teszi a rendszert, mert minden komponensnek meghatározott feladata van.

4.3 Adatmodell

A rendszer adatmodellje a felhasználóhoz kapcsolódó beszélgetések, üzenetek és mappák kezelésére épül. Mivel az alkalmazás bejelentkezéshez kötött matematikai tutor funkciót biztosít, fontos, hogy minden tanulási előzmény egy konkrét felhasználóhoz tartozzon. Emiatt az adatmodell központi eleme a felhasználó, amelyhez a beszélgetések és a mappák kapcsolódnak.

A felhasználót a *User* entitás írja le. Ehhez tartoznak a felhasználó azonosításához szükséges adatok, például az egyedi azonosító, az e-mail cím, a jelszó tárolásához használt mező, valamint a létrehozás időpontja. Egy felhasználó több beszélgetést és több mappát is létrehozhat, ezért a rendszerben a *User* entitás kapcsolódik a *Chat* és a *Folder* entitásokhoz.

A *Chat* entitás egy matematikai tutor beszélgetést jelöl. Egy beszélgetés mindig egy felhasználóhoz tartozik és opcionálisan egy mappába is besorolható. A chathez tartozik többek között egy cím, a létrehozás és módosítás időpontja, valamint egy állapotjelző mező, amely azt mutatja, hogy a beszélgetés befejezettnek tekinthető-e. A *Chat* entitás szerepe az, hogy egy tanulási folyamat üzeneteit egy egységbe fogja össze.

A *ChatMessage* entitás a beszélgetésen belüli egy konkrét üzenetet reprezentál. Ez lehet a felhasználó által beküldött kérdés vagy az AI tutor által adott válasz. Az üzenethez tartozik a küldő szerepe, az üzenet szöveges tartalma, valamint egy kategória, amely segít megkülönböztetni például a feladatgenerálást, a problémamegoldást, a folytató kérdést vagy a nem matematikai kérést. Ez azért hasznos, mert a rendszer nem csak egyszerű szöveges üzenetként kezeli a válaszokat, hanem a tutor működéséhez kapcsolódó állapotinformációkat is tárol.

A *Folder* entitás a beszélgetések rendszerezésére szolgál. Egy felhasználó több mappát is létrehozhat és ezekhez beszélgetéseket rendelhet. Ez a megoldás egyszerűbbé teszi a korábbi beszélgetések áttekintését, különösen akkor, ha a felhasználó több különböző matematikai témában használja az alkalmazást. A mappák tehát nem teljes tananyagkezelő rendszerként működnek, hanem a mentett beszélgetések csoportosítását támogatják.

Az adatmodellben szerepel egy korábbi Conversation nevű modell is, amely a régebbi, determinisztikus demo működéshez kapcsolódott. A jelenlegi chat-alapú tutor működésben ez már nem kap szerepet, ezért legacy elemként kezelhető. A ténylegesen használt adatmodell központi részeit a *User*, *Chat*, *ChatMessage* és *Folder* entitások adják.

Az adatmodell fő kapcsolatait az *M4.3.1-es mellékletben* található adatmodell-ábra szemlélteti. Az ábra alapján látható, hogy a *User* entitás áll az adatmodell középpontjában. Ehhez kapcsolódnak a felhasználó beszélgetései és mappái. A *Chat* entitás több *ChatMessage* rekordot tartalmaz, így egy beszélgetés teljes előzménye üzenetszinten tárolható. A *Folder* entitás a beszélgetések csoportosítását biztosítja és ezzel segíti a felhasználót a korábbi tanulási folyamatok áttekintésében.

4.4 API-végpontok áttekintése

A frontend és a backend közötti kommunikáció REST API-végpontokon keresztül történik. A felhasználó a webes felületen végez műveleteket, például bejelentkezik, új beszélgetést indít, üzenetet küld vagy mappát hoz létre. Ezek a műveletek a háttérben HTTP-kérések formájában jutnak el a backendhez, amely feldolgozza a kérést, szükség esetén adatbázisműveletet hajt végre, majd választ küld vissza a frontendnek.

Az API-végpontok négy fő csoportba sorolhatók. Ezek közül az első a technikai ellenőrzéshez kapcsolódó Health végpont, amely a backend elérhetőségének vizsgálatára szolgál. Ez nem közvetlen felhasználói funkció, hanem a rendszer futásának és alapvető elérhetőségének ellenőrzését segíti.

A második csoportot az autentikációhoz kapcsolódó végpontok alkotják. Ezek kezelik a regisztrációt, a bejelentkezést, az aktuális felhasználó lekérését, a jelszó módosítását és a fiók törlését. Ezek a végpontok biztosítják, hogy a rendszer a felhasználóhoz kötöten tudja kezelni a beszélgetéseket, mappákat és profiladatokat.

A harmadik csoportba a chathez és a tutor működéshez kapcsolódó végpontok tartoznak. Ezek teszik lehetővé új beszélgetés indítását, meglévő beszélgetés folytatását, a beszélgetések listázását, egy konkrét beszélgetés megnyitását, valamint a beszélgetések törlését. A matematikai tutor működése szempontjából ezek a végpontok központi szerepet töltenek be, mivel a felhasználói üzenetek ezeken keresztül jutnak el a backendhez, majd az AI tutor modulhoz.

A negyedik csoportot a mappakezeléshez tartozó végpontok alkotják. Ezek segítségével a felhasználó mappákat hozhat létre, lekérheti a meglévő mappáit, illetve törölheti azokat. A mappák a beszélgetések rendszerezését támogatják, így az API ezen része a tanulási előzmények áttekinthetőbb kezeléséhez kapcsolódik.

API-csoport	Feladat	Példa végpont
<i>Health</i>	A backend elérhetőségének ellenőrzése	GET /health
<i>Auth</i>	Regisztráció, bejelentkezés és felhasználói fiók kezelése	POST /api/auth/login
<i>Chat és tutor</i>	Beszélgetés indítása, folytatása és lekérése	POST /api/qa/chats/start
<i>Mappakezelés</i>	Mappák létrehozása, listázása és törlése	POST /api/qa/folders

4.4.1.ábra: Az API-végpontok fő csoportjai

4.5 AI tutor integráció

Az AI tutor integráció a backendhez kapcsolódó működési egységként jelenik meg a rendszerben. A felhasználó nem közvetlenül a nyelvi modellel kommunikál, hanem a webes felületen keresztül küld üzenetet. A frontend ezt az üzenetet API-kérésként továbbítja a backend felé, amely a szükséges ellenőrzések és adatkezelési lépések után hívja meg az AI tutor modult.

A folyamat egy matematikai kérdés vagy feladat beküldésével indul. A backend először ellenőrzi a felhasználó hitelesítését, majd létrehozza vagy frissíti az adott beszélgetést. Ezt követően eltárolja a felhasználó üzenetét, majd továbbítja a kérdést az AI tutor modulnak. A tutor által generált válasz visszakerül a backendhez, amely az asszisztens válaszát is menti az adatbázisban, majd továbbítja azt a frontend számára megjelenítésre.

Az AI tutor modul szerepe nem egyszerűen egy általános válasz előállítás. A működését rendszerutasítás szabályozza, amely meghatározza, hogy a modell matematikai témára fókuszáljon, tutor jellegű választ adjon és a nem matematikai kéréseket elutasítsa. A rendszer célja, hogy a felhasználó ne minden esetben azonnal teljes megoldást kapjon, hanem olyan segítséget, amely a következő lépésre vagy a feladat megértésére irányítja a figyelmét.

A modul a beérkező kéréseket különböző típusok szerint kezelheti. Ilyen lehet például a feladatgenerálás, a problémamegoldás, a folytató kérdés vagy a nem matematikai kérdés. Feladatgenerálás esetén a cél egyetlen matematikai feladat létrehozása, megoldás nélkül. Problémamegoldásnál és folytató kérdésnél a rendszer rövid, tanulást támogató választ ad. Nem matematikai kérdés esetén pedig a rendszer jelzi, hogy csak matematikai kérdések és feladatok támogatására használható.

Az AI tutor válasza nemcsak szöveges tartalomként jelenik meg, hanem a beszélgetés részeként is eltárolódik. Ez azért fontos, mert így a felhasználó később visszatérhet a korábbi tanulási folyamatához. A mentett üzenetek alapján a beszélgetés nem egyszeri kérdés-válasz formában működik, hanem folytatható párbeszédként jelenik meg.

A nem matematikai kérések kezelése az integráció egyik fontos része. Mivel a használt nyelvi modell önmagában sokféle témában képes lenne választ adni, a rendszernek korlátoznia kell a működést a választott alkalmazási területre. Emiatt a nem matematikai jellegű kérések esetén a tutor elutasító választ ad. Ez segít abban, hogy az

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

alkalmazás ne általános chatbotként működjön, hanem megmaradjon matematikai tanulástámogató rendszerként.

Az integráció során a hibás modellállapotokat is kezelni kell. Ilyen eset lehet például a hiányzó Hugging Face token, a sikertelen modellbetöltés vagy az erőforráshiány. Ezekben az esetekben a rendszer nem váratlan leállással reagál, hanem a válaszban jelzi, hogy a tutor nincs megfelelően konfigurálva vagy átmenetileg nem elérhető. Ez lehetővé teszi, hogy az alkalmazás többi része, például a bejelentkezés, a beszélgetések mentése és a mappakezelés továbbra is ellenőrizhető maradjon.

Az AI tutor integráció tehát a frontend, a backend, az adatbázis és a modell közötti együttműködésre épül. A frontend biztosítja a felhasználói felületet, a backend végzi az ellenőrzést és az adatkezelést, az adatbázis tárolja a beszélgetések előzményeit, míg az AI tutor modul a matematikai segítségnyújtásért felel. Ez az elkülönített működés átláthatóbbá teszi a rendszert és lehetőséget ad arra, hogy az AI-alapú rész később külön is továbbfejleszthető legyen.

5. Megvalósítás

A megvalósítás során a korábban bemutatott követelmények és rendszerterv alapján készült el az alkalmazás frontendje, backendje, adatkezelése és AI tutor modulja. A fejezetben azok a főbb megoldások szerepelnek, amelyek az alkalmazás működése szempontjából meghatározók.

A rendszer külön frontend és backend komponensből áll. A frontend biztosítja a felhasználói felületet, míg a backend kezeli a hitelesítést, az adatbázisműveleteket, a chatfolyamatokat és az AI tutor modul meghívását. A megvalósítás bemutatása a projekt szerkezetével indul, majd a felhasználókezelés, a chatfunkció, a mappák, a profilfunkciók, valamint az AI tutor működése kerül ismertetésre.

5.1 Projektstruktúra

A projekt felépítése úgy lett kialakítva, hogy a frontend, a backend, a dokumentáció és a tesztek elkülönüljenek egymástól. Ez átláthatóbbá teszi a fejlesztést, mert a felhasználói felülethez, a szerveroldali működéshez és a projekt leírásához tartozó fájlok külön helyen találhatók.

A frontend mappában található a Next.js^[2] alapú webes felület, amely a bejelentkezési és regisztrációs oldalakért, a fő chatfelületért, a mappakezelésért és a profilhoz kapcsolódó képernyőkért felel. A backend mappa tartalmazza a FastAPI^[4] alkalmazást, az API-végpontokat, az adatmodelleket, az adatbázis-kezeléshez kapcsolódó részeket, valamint az AI tutor modult.

A projektben külön dokumentációs mappa is szerepel, amelyben az architektúra, az API-végpontok, az adatmodell, a reprodukálható futtatás, a tesztelés és az AI tutor használata van dokumentálva. Ez azért fontos, mert a projekt nemcsak forráskódból áll, hanem a működés megértését és ellenőrzését segítő háttéranyagokból is.

A lokális futtatást a *docker-compose.yml* fájl támogatja, amely az adatbázis és a backend indítását segíti. A környezeti változókhoz példafájlok tartoznak, amelyek megadják a szükséges beállításokat, például az adatbázis elérését, a backend titkos kulcsát vagy az AI modellhez szükséges tokenet.

Mappa / fájl	Szerepe
frontend/	A Next.js alapú felhasználói felület forráskódja
backend/	A FastAPI backend, az API-végpontok, az adatmodellek és az AI tutor modul
docs/	A projekt dokumentációi, például architektúra, API, adatmodell, tesztelés és reprodukció
tests/	Tesztek és acceptance jellegű fájlok
docker-compose.yml	Az adatbázis és a backend lokális indításának támogatása
.env.example fájlok	A szükséges környezeti változók mintái

5.1.1.ábra: A projekt főbb mappái és fájljai

5.2 Felhasználókezelés megvalósítása

A felhasználókezelés backend oldalon az autentikációs végpontokon keresztül valósul meg. Ezek a végpontok a `/api/auth` útvonal alatt érhetők el és a regisztrációt, a bejelentkezést, az aktuális felhasználó lekérését, a jelszó módosítását és a fiók törlését kezelik.

A regisztráció során a backend ellenőrzi, hogy a megadott e-mail cím szerepel-e már az adatbázisban. Ha igen, akkor hibát ad vissza, ellenkező esetben létrehozza az új felhasználót. A jelszó nem eredeti formájában kerül tárolásra, hanem hash-elt változatban, amelyet a rendszer a biztonsági segédfüggvényeken keresztül állít elő.

A bejelentkezés a `POST /api/auth/login` végponton keresztül történik. A backend először az e-mail-cím alapján megkeresi a felhasználót, majd ellenőrzi, hogy a megadott jelszó megfelel-e az adatbázisban tárolt hash-nek. Ha a felhasználó nem létezik vagy a jelszó hibás, a rendszer 401-es hibát ad vissza. Sikeres bejelentkezés esetén a backend JWT^[25] tokenet hoz létre, amelyet a frontend a későbbi védett kérésekhez használ.

A token létrehozásánál a rendszer a felhasználó azonosítóját helyezi el a token subject mezőjében, valamint lejáratí időt is beállít. A jelenlegi megvalósításban a bejelentkezéskor kiadott token 60 perces lejáratú készül. A token aláírásához a backend a konfigurációban megadott titkos kulcsot használja.

A védett végpontok elérésekor a backend a kérésben kapott Bearer tokenet ellenőrzi. A token dekódolása után a rendszer kiolvassa belőle a felhasználó azonosítóját, majd ez alapján lekéri az aktuális felhasználót az adatbázisból. Ha a token

hiányzik, érvénytelen vagy a hozzá tartozó felhasználó nem található, a rendszer jogosultsági hibát ad vissza.

A profilhoz kapcsolódó műveletek szintén védett végpontokon keresztül érhetők el. Az aktuális felhasználó adatai a `/api/auth/me` végponton kérhetők le, a jelszó módosítása a `/api/auth/me/password` végponton történik, a fiók törlése pedig a `DELETE /api/auth/me` végponton keresztül valósul meg. A jelszó módosításánál a rendszer ellenőrzi a régi jelszót, valamint azt is, hogy az új jelszó ne egyezzen meg a régivel.

A felhasználókezelés megvalósítása így nemcsak a regisztrációt és bejelentkezést biztosítja, hanem a védett funkciók alapját is adja. A beszélgetések, mappák és profiladatok csak érvényes tokennel érhetők el, ezért a hitelesítés központi szerepet tölt be az alkalmazás működésében.

```
@router.post("/login", response_model=TokenResponse)
def login(payload: LoginRequest, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == payload.email).first()
    if not user or not verify_password(payload.password,
    user.password_hash):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid credentials")
    access_token = create_access_token(subject=str(user.id),
    expires_delta=timedelta(minutes=60))
    return TokenResponse(access_token=access_token)
```

5.2.1.kódrészlet: A bejelentkezési végpont egyszerűsített működése

5.3 Chatfunkció és beszélgetések kezelése

A chatfunkció megvalósítása a backend oldalon a `/api/qa` útvonalhoz tartozó végpontokon keresztül történik. A rendszer külön kezeli az új beszélgetés indítását, a meglévő beszélgetés folytatását, a beszélgetések listázását, egy konkrét beszélgetés lekérését, valamint a beszélgetés törlését. Ezek a műveletek minden esetben bejelentkezett felhasználóhoz kapcsolódnak, ezért a backend a védett végpontoknál az aktuális felhasználót is ellenőrzi.

Új beszélgetés indításakor a frontend elküldi a felhasználó első üzenetét a backendnek. A backend létrehoz egy új *Chat* rekordot, amelyet az aktuális felhasználóhoz rendel, majd eltárolja a felhasználó üzenetét *ChatMessage* rekordként. Ezután meghívja az AI tutor modult, amely választ készít a beküldött matematikai kérdésre. A kapott válasz szintén *ChatMessage* rekordként kerül mentésre, asszisztensi

szereppel. Így már a beszélgetés kezdetén eltárolódik mind a felhasználói kérdés, mind a rendszer válasza.

Meglévő beszélgetés folytatásakor a backend először ellenőrzi, hogy az adott chat létezik-e és valóban az aktuális felhasználóhoz tartozik-e. Ez azért fontos, mert a rendszernek biztosítania kell, hogy a felhasználók csak a saját beszélgetéseiket érhessék el. Ha a beszélgetés megtalálható, a rendszer eltárolja az új felhasználói üzenetet, összeállítja a beszélgetési előzményt, majd ezt továbbítja az AI tutor modulnak. A tutor válasza ezután ugyanahhoz a beszélgetéshez kapcsolódva kerül mentésre.

A beszélgetések listázásakor a rendszer az aktuális felhasználóhoz tartozó chateket kéri le az adatbázisból, a legutóbb frissített beszélgetéseket előre sorolva. A lista nem a teljes üzenetelőzményt tartalmazza, hanem egy rövidebb áttekintést ad a beszélgetésekről. Ilyen adat például a beszélgetés címe, az azonosítója, az esetleges mappazonosító, a befejezettségi állapota, valamint egy rövid előnézet az utolsó asszisztensi válaszból.

Egy konkrét beszélgetés megnyitásakor a backend a teljes beszélgetést adja vissza, az ahhoz tartozó üzenetekkel együtt. Ez teszi lehetővé, hogy a felhasználó később visszatérjen egy korábbi matematikai feladathoz és ott folytassa a tanulási folyamatot, ahol korábban abbahagyta. A beszélgetés törlésekor a rendszer szintén ellenőrzi, hogy a chat az aktuális felhasználóhoz tartozik-e, majd sikeres ellenőrzés után eltávolítja azt az adatbázisból.

A chatfunkció megvalósításának lényege, hogy a felhasználói és asszisztensi üzenetek nem különálló válaszokként, hanem egy beszélgetés részeként tárolódnak. Ez biztosítja, hogy az alkalmazás ne csak egyszeri kérdés-válasz működésre legyen alkalmas, hanem valódi, folytatható tutor jellegű párbeszédet tudjon támogatni.

5.4 Mappakezelés és profilműködések megvalósítása

A mappakezelés célja, hogy a felhasználó a korábbi beszélgetéseit egyszerűen rendszerezni tudja. A megvalósítás backend oldalon a `/api/qa` útvonalhoz kapcsolódó végpontokon keresztül történik. A felhasználó új mappát hozhat létre, lekérheti a meglévő mappáit, beszélgetést rendelhet egy adott mappához, illetve törölhet mappát.

Új mappa létrehozásakor a backend az aktuális felhasználóhoz kapcsolva hozza létre a Folder rekordot. Ez biztosítja, hogy a mappák felhasználónként elkülönüljenek egymástól. A mappák listázásakor a rendszer csak az adott bejelentkezett

felhasználóhoz tartozó mappákat adja vissza, így más felhasználók mappái nem jelenhetnek meg a felületen.

A beszélgetések mappába rendezése külön műveletként valósul meg. Ilyenkor a backend először ellenőrzi, hogy a kiválasztott beszélgetés az aktuális felhasználóhoz tartozik-e. Ha a felhasználó mappát is megad, akkor a rendszer azt is ellenőrzi, hogy a mappa valóban az adott felhasználóhoz kapcsolódik-e. Csak ezek után történik meg a beszélgetés mappához rendelése. Ez a működés azért fontos, mert így a mappakezelés nem kerülheti meg a jogosultsági ellenőrzést.

A mappa törlésekor a backend szintén ellenőrzi, hogy a törölni kívánt mappa az aktuális felhasználóhoz tartozik-e. Ha a mappa nem található vagy nem a bejelentkezett felhasználóhoz kapcsolódik, a rendszer hibát ad vissza. A mappakezelés így a chatfunkció kiegészítő részeként működik, vagyis nem önálló tananyagkezelő rendszer, hanem a mentett beszélgetések áttekinthetőbb rendszerezését támogatja.

A profilfunkciók a felhasználói fiók alapvető kezelését biztosítják. Az aktuális felhasználó adatai a `/api/auth/me` végponton keresztül kérhetők le. Ez a végpont védett, vagyis csak érvényes tokennel érhető el. A backend a token alapján azonosítja a felhasználót, majd visszaadja a profilhoz szükséges alapadatokat.

A jelszó módosítása a `/api/auth/me/password` végponton keresztül történik. A művelet során a rendszer ellenőrzi, hogy az új jelszó eltér-e a régitől, majd megvizsgálja, hogy a megadott régi jelszó valóban megfelel-e a tárolt jelszóhash-nek. Sikeres ellenőrzés után az új jelszó hash-elt formában kerül mentésre. Ez biztosítja, hogy a jelszó módosítása ellenőrzött módon történjen.

A fiók törlése a `DELETE /api/auth/me` végponton keresztül valósul meg. Ez a művelet szintén az aktuális bejelentkezett felhasználóhoz kötődik. A frontend oldalon ehhez megerősítés kapcsolódik, mivel a fióktörlés érzékeny művelet és a felhasználóhoz tartozó adatok elvesztésével járhat.

A mappakezelés és a profilfunkciók megvalósítása így a személyes használatot támogatja. A felhasználó nemcsak matematikai beszélgetéseket indíthat és folytathat, hanem rendszerezheti is azokat, valamint kezelheti a saját fiókjához tartozó alapvető műveleteket.

5.5 AI tutor működése

Az AI tutor működése backend oldalon külön modulban valósul meg. A modul feladata, hogy a felhasználó matematikai jellegű kérdésére vagy feladatára rövid, tutor jellegű választ adjon. A rendszer nem közvetlenül a felhasználói felületről hívja meg a modellt, hanem a backend chatfolyamatán keresztül. Így a felhasználói üzenet először eltárolódik, majd az AI tutor válasza is ugyanahhoz a beszélgetéshez kapcsolódva kerül mentésre.

A tutor működésének alapját egy rendszerutasítás adja meg. Ez határozza meg, hogy a modell angol nyelven válaszoljon, csak matematikai jellegű kérésekkel foglalkozzon és tutor stílusban működjön. A cél nem az, hogy a rendszer minden esetben azonnal teljes megoldást adjon, hanem az, hogy a tanulót a következő lépés vagy egy rövid magyarázat segítségével vezesse tovább.

A modul a beérkező üzenetet először ellenőrzi abból a szempontból, hogy kapcsolódik-e matematikához. Ehhez egyszerű heurisztikákat használ, például matematikai kulcsszavakat, műveleti jeleket, egyenletekre utaló mintákat vagy ismert függvényneveket keres. Ha a kérés nem matematikai jellegű és az előzményekben sincs matematikai kontextus, akkor a rendszer nem hívja meg a modellt, hanem egy előre meghatározott elutasító választ ad vissza.

A matematikai jellegű kéréseknél a rendszer megpróbálja meghatározni a felhasználói szándékot is. Külön kezeli például az új feladat generálását, a problémamegoldást, a folytató kérdéseket, a végső válasz kérését és a felhasználó által megadott válasz ellenőrzését. Ez azért fontos, mert a tutor válaszáinak stílusa az adott helyzettől függ. Egy feladatgenerálási kérésnél a rendszernek egyetlen feladatot kell adnia megoldás nélkül, míg egy folytató kérdésnél a korábbi beszélgetéshez igazodó következő lépést kell megfogalmaznia.

A modell betöltése csak akkor történik meg, ha a szükséges konfiguráció rendelkezésre áll. A Hugging Face token hiánya esetén a rendszer nem próbálja meg betölteni a modellt, hanem konfigurációs figyelmeztetést ad vissza. Ha a modell betöltése vagy futtatása közben hiba történik, a rendszer átmeneti elérhetetlenséget jelző választ küld. Ez biztosítja, hogy a felhasználó ne váratlan hibával találkozzon, hanem érthető visszajelzést kapjon.

A modell választ a rendszer lehetőség szerint strukturált JSON formátumban^[31] dolgozza fel. A válasz kategóriát, befejezettségi állapotot és szöveges választ

tartalmazhat. Ha a modell nem ad megfelelő JSON-t^[31], akkor a rendszer megpróbálja a nyers szövegből kinyerni a használható választ. Ez a fallback működés azért hasznos, mert a nyelvi modell nem minden esetben követi pontosan az elvárt válaszformátumot.

A tutor modul további utófeldolgozást is végez. A feladatgenerálásnál igyekszik kiszűrni a megoldást vagy túl hosszú magyarázatot, problémamegoldásnál és folytató kérdéseknél pedig rövidebb, egy lépésre koncentráló választ próbál kialakítani. A rendszer emellett normalizálja a matematikai jelöléseket is, hogy azok a frontend oldalon KaTeX^[23] segítségével olvasható formában jelenhessenek meg.

Az AI tutor működése tehát több lépésből áll, kezdésként a kérés előszűréséből, a szándék felismeréséből, a modell szükség szerinti meghívásából, végül a válasz feldolgozásából és az eredmény visszaadásából. Ez a megoldás segít abban, hogy az alkalmazás ne általános chatbotként működjön, hanem a matematikai tanulási folyamatot támogató tutor szerepben maradjon.

```
if not _looks_math_related(user_message) and not
    _looks_math_related(history_context):
    return {
        "category": "non_math",
        "problem_completed": False,
        "reply": MATH_SCOPE_REFUSAL,
    }

intent = _classify_intent(user_message, history_context)
category_intent = "follow_up" if intent in {"answer_request",
"answer_check"} else intent

if not settings.hf_token:
    return {
        "category": "problem_solving",
        "problem_completed": False,
        "reply": "The math tutor model is not configured yet. Set HF_TOKEN
on the backend to enable Qwen tutoring.",
    }
```

5.5.1.kódrészlet: A tutor bemenetének előszűrése és konfigurációellenőrzése

5.6 Hibakezelés és ismert korlátok

A megvalósítás során fontos szempont volt, hogy a rendszer a hibás vagy sikertelen műveleteket kezelhető módon jelezze. Ez különösen lényeges egy olyan alkalmazásnál, ahol a felhasználó bejelentkezéshez kötött műveleteket végez, adatokat ment, korábbi beszélgetéseket nyit meg, valamint AI-alapú választ vár a rendszertől.

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

A felhasználókezeléshez kapcsolódó hibák közé tartozik például, ha a regisztráció során a megadott e-mail-cím már szerepel az adatbázisban vagy ha a bejelentkezésnél hibás e-mail-cím vagy jelszó kerül megadásra. Ezekben az esetekben a backend megfelelő HTTP-hibát ad vissza, amelyet a frontend felhasználói visszajelzésként tud megjeleníteni. A jelszó módosításánál a rendszer külön ellenőrzi azt is, hogy a régi jelszó helyes-e, illetve hogy az új jelszó ne egyezzen meg a korábbival.

A védett végpontoknál a rendszer a JWT token alapján azonosítja a felhasználót. Ha a token hiányzik, érvénytelen vagy olyan felhasználóra hivatkozik, amely nem található az adatbázisban, a backend jogosultsági hibát ad vissza. Ez biztosítja, hogy a felhasználó csak bejelentkezés után érhesse el a saját beszélgetéseit, mappáit és profiladatait.

A chat- és mappakezelésnél a hibakezelés egyik fontos része az erőforrások ellenőrzése. Egy beszélgetés megnyitása, folytatása, törlése vagy mappába helyezése előtt a backend ellenőrzi, hogy az adott chat valóban az aktuális felhasználóhoz tartozik-e. Hasonló ellenőrzés történik a mappák esetében is. Ha a keresett beszélgetés vagy mappa nem található, a rendszer nem hajtja végre a műveletet, hanem hibaválaszt ad.

Az AI tutor működésénél külön hibaágak is megjelennek. Ha a felhasználó nem matematikai jellegű kérést küld, a rendszer nem általános chatbotként válaszol, hanem elutasító üzenetet ad vissza. Ha a modell használatához szükséges Hugging Face token nincs beállítva, a tutor jelzi, hogy a modell még nincs megfelelően konfigurálva. Ha a modell betöltése vagy futtatása közben hiba történik, a rendszer átmeneti elérhetetlenséget jelző választ ad vissza.

A rendszer jelenlegi állapotában prototípusként értelmezhető, ezért több korláttal is rendelkezik. Az AI válaszai nem tekinthetők minden esetben hibátlannak, ezért a matematikai eredmények ellenőrzése továbbra is szükséges. A modell futtatása erőforrás-igényes lehet, különösen CPU használata esetén. További korlát, hogy a rendszer nem tartalmaz teljes körű éles üzemi biztonsági megoldásokat, részletes jogosultsági naplózást, felhasználói szintű rate limitet vagy fejlett adminisztrációs felületet.

Összességében a hibakezelés célja az volt, hogy a rendszer a legfontosabb hibás működési helyzetekben ne váratlanul álljon le, hanem értelmezhető választ adjon a

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

felhasználónak. Az ismert korlátok ugyanakkor kijelölik azokat a területeket is, amelyek egy későbbi, élesebb használatra szánt változatban továbbfejlesztést igényelhetnek.

6. Tesztelés és validáció

A fejlesztés során fontos szempont volt annak ellenőrzése, hogy az alkalmazás fő funkciói stabilan és a tervezett működésnek megfelelően működjenek. A tesztelés kiterjedt a backend API-végpontokra, a frontend felhasználói működésére, valamint az AI tutor alapvető viselkedésére is.

A projekt fejlesztése során folyamatos manuális és funkcionális ellenőrzés történt. A vizsgálatok célja annak ellenőrzése volt, hogy a rendszer megfelelően kezeli-e a felhasználókezelést, a beszélgetések működését, a mappakezelést, valamint az AI tutor válaszait és hibakezelését.

A backend tesztelése során ellenőrzésre kerültek az autentikációs végpontok, a chatkezeléshez kapcsolódó API-műveletek, valamint a mappakezelési funkciók. A frontend oldalon a felhasználói felület működése, a beszélgetések kezelése, a matematikai válaszok megjelenítése és a hibajelzések ellenőrzése történt meg. Az AI tutor validációja során külön figyelmet kapott a matematikai és nem matematikai kérések elkülönítése, valamint a tutor válaszainak konzisztenciája.

6.1 Backend és API tesztelése

A backend és az API-végpontok tesztelésének célja annak ellenőrzése volt, hogy a rendszer megfelelően kezeli-e a felhasználói műveleteket, az adatbázis-műveleteket, valamint az AI tutorhoz kapcsolódó folyamatokat. A tesztelés során külön ellenőrzésre kerültek az autentikációs végpontok, a chatkezelési funkciók, a mappakezelés és a jogosultsági ellenőrzések.

A tesztelés elsősorban manuális API-hívásokkal történt. Az egyes végpontok működésének ellenőrzésére a FastAPI^[4] által biztosított Swagger felület is használható volt, amely lehetővé tette a különböző HTTP-kérések közvetlen kipróbálását. A vizsgálatok során nemcsak a sikeres működés, hanem a hibás bemenetek és jogosultsági helyzetek kezelése is ellenőrzésre került.

A felhasználókezelés tesztelése során ellenőrzésre került a regisztráció, a bejelentkezés, a JWT token kiadása, valamint a védett végpontok működése. Vizsgálatra kerültek azok az esetek is, amikor a felhasználó hibás jelszót adott meg, már létező

e-mail-címmel próbált regisztrálni vagy érvénytelen tokennel próbált védett műveletet végrehajtani.

A chatfunkció tesztelése során ellenőrzésre került az új beszélgetések létrehozása, az üzenetküldés, a beszélgetések listázása, valamint a korábbi beszélgetések lekérése és törlése. A vizsgálatok során külön figyelmet kapott annak ellenőrzése, hogy a felhasználók csak a saját beszélgetéseiket érhessék el. Hasonló ellenőrzések történtek a mappakezelési funkcióknál is.

Az AI tutor működéséhez kapcsolódó API-hívásoknál ellenőrzésre került, hogy a rendszer megfelelően menti-e a felhasználói és asszisztensi üzeneteket, valamint hogy a backend hibakezelése megfelelő választ ad-e modellhiba vagy konfigurációs probléma esetén.

A backend és API működésének ellenőrzése során a legfontosabb funkciók sikeresen működtek és a rendszer a vizsgált hibás eseteknél is kezelhető válaszokat adott vissza.

Funkció	Tesztelt működés	Eredmény
Regisztráció	Új felhasználó létrehozása	Sikeres
Bejelentkezés	JWT token kiadása	Sikeres
Chat indítása	Új beszélgetés létrehozása	Sikeres
Üzenetküldés	AI válasz mentése	Sikeres
Mappakezelés	Chat mappába helyezése	Sikeres
Jogosultság	Más felhasználó chatjének tiltása	Sikeres

6.1.1. ábra: A backend és API főbb funkcióinak ellenőrzése

6.2 Frontend és felhasználói működés ellenőrzése

A frontend tesztelésének célja annak ellenőrzése volt, hogy a felhasználói felület megfelelően együttműködik-e a backend API-val, valamint hogy a fő funkciók a felhasználó számára stabilan és áttekinthető módon legyenek használhatók.

A tesztelés során a regisztráció és a bejelentkezés, a beszélgetések indítása és folytatása, a mappakezelés, valamint a profilfunkciók működése került előtérbe. Fontos szempont volt az is, hogy az állapotváltozások megfelelően jelenjenek meg a felhasználói felületen, például új üzenet küldése, beszélgetés törlése vagy mappába helyezés után.

A chatfelület esetében ellenőrizni kellett, hogy a felhasználói és asszisztensi üzenetek megfelelő sorrendben jelennek-e meg, valamint hogy a korábbi beszélgetések

visszatöltése megfelelően működik-e. A matematikai válaszok megjelenítésénél külön figyelmet kapott a KaTeX-alapú^[23] képletrenderelés, hogy a matematikai jelölések olvasható formában jelenjenek meg a felhasználói felületen.

A frontend működésének részeként a hibakezelés is tesztelésre került. Ilyen eset például a hibás bejelentkezési adat, az érvénytelen token, a sikertelen API-hívás vagy az AI tutor átmeneti elérhetetlensége. A rendszer ezekben az esetekben felhasználói visszajelzést jelenít meg, amely segíti a felhasználót a probléma azonosításában.

Az alkalmazás kialakítása során fontos szempont volt az egyszerű és áttekinthető felhasználói működés. A frontend ellenőrzése ezért nemcsak a funkcionális működésre, hanem a használhatóságra is kiterjedt. A beszélgetések kezelése, a navigáció és a válaszok megjelenítése a tesztelések során stabilan működött.

6.3 AI tutor működésének validációja

Az AI tutor validációjának célja annak ellenőrzése volt, hogy a rendszer a matematikai kérdésekre a tervezett tutor jellegű működés szerint válaszol-e. A vizsgálat során nem az volt az elsődleges szempont, hogy a modell minden esetben tökéletes matematikai választ adjon, hanem az, hogy a válaszok illeszkedjenek az alkalmazás céljához, vagyis legyenek rövidek, matematikai fókuszúak és támogassák a tanulási folyamatot.

A validáció manuális teszteléssel történt a webes felületen keresztül. A tesztek között szerepelt feladatgenerálás, konkrét matematikai kérdés megoldásának támogatása, végső válasz kérése, felhasználói válasz ellenőrzése, valamint nem matematikai kérés kezelése is. Ezek a tesztek azért voltak fontosak, mert az alkalmazás célja nem egy általános chatbot létrehozása, hanem egy matematikai tutor jellegű segédeszköz megvalósítása.

A tesztelés során a feladatgenerálásnál kezdetben előfordult, hogy a modell nem közvetlenül matematikai feladatot adott vissza, hanem rövid meta jellegű magyarázatot kezdett generálni arról, hogy milyen feladatot fog létrehozni. Emiatt a rendszerutasítás és a feladatgenerálási válaszok utófeldolgozása pontosításra került. A módosítás után a rendszer már egyetlen konkrét matematikai feladatot adott vissza megoldás nélkül, amely megfelelt az elvárt működésnek.

A matematikai kérdések esetén az AI tutor értelmezhető, rövid választ adott és jellemzően nem a teljes megoldást közölte azonnal, hanem egy következő lépést vagy

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

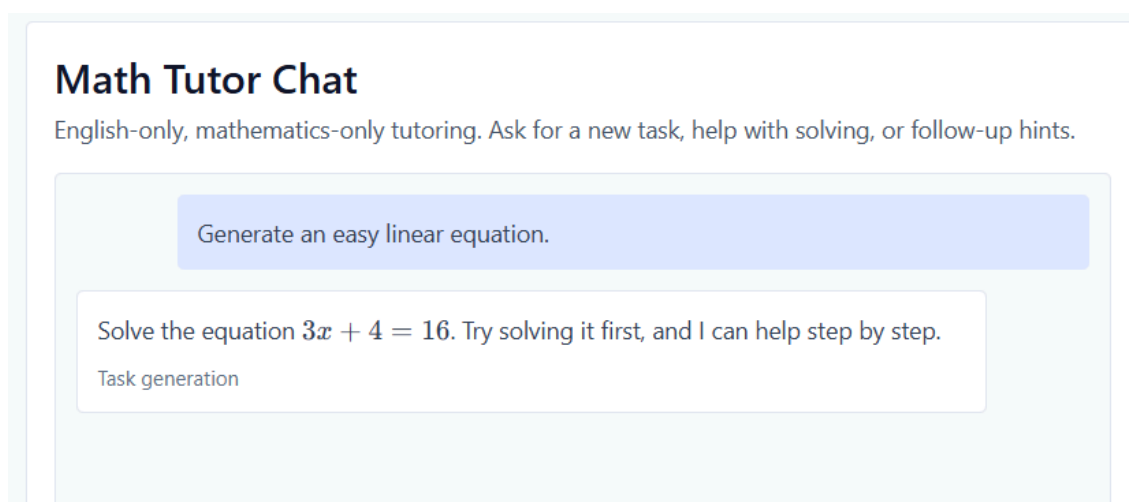
iránymutatást fogalmazott meg. Ez megfelel annak az elvárásnak, hogy a rendszer ne egyszerű megoldásgenerátorként működjön, hanem a tanulási folyamatot segítse.

A nem matematikai kérdés tesztelésekor a rendszer elutasító választ adott. Ez különösen fontos, mert a használt nyelvi modell önmagában sokféle témában képes lenne választ generálni, az alkalmazás hatóköre viszont kizárólag matematikai kérdések és feladatok támogatására terjed ki.

Teszt eset	Elvárt működés	Eredmény
<i>Feladatgenerálás</i>	Egy konkrét matematikai feladat generálása megoldás nélkül	Sikeres
<i>Matematikai kérdés</i>	Tutor jellegű segítség adása	Sikeres
<i>Végző válasz kérdés</i>	Ne adjon teljes megoldást, maradjon tutor szerepben	Sikeres
<i>Válaszellenőrzés</i>	Ellenőrzés vagy további lépések kérése	Sikeres
<i>Nem matematikai kérdés</i>	Elutasító válasz adása	Sikeres

6.3.1. ábra: Az AI tutor manuális validációjának eredményei

A tesztek alapján az AI tutor a vizsgált esetekben az elvárt működésnek megfelelő választ adott. A feladatgenerálásnál az első validáció során talált hiba javítása után a rendszer már megfelelően működött, ahol egy konkrét matematikai feladatot jelenített meg, megoldási lépések nélkül.



6.3.2. ábra: Példa az AI tutor feladatgenerálási válasza

Az ábrán látható példa alapján a tutor egy konkrét lineáris egyenletet generál, de nem adja meg hozzá a megoldást. Ez megfelel annak az elvárásnak, hogy feladatgenerálás

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

esetén a rendszer a felhasználónak adjon gyakorlófeladatot, majd a megoldási folyamatban csak később, kérdés alapján segítsen.

A validáció eredményei alapján az AI tutor a vizsgált esetekben megfelelt az alkalmazással szemben támasztott alapvető működési elvárásoknak. A rendszer képes volt matematikai feladatot generálni, konkrét matematikai kérdésre tutor jellegű segítséget adni, a végső válasz azonnali közlése helyett a tanulási folyamatot támogatni, valamint a nem matematikai kéréseket elutasítani. Az AI-alapú működés miatt ugyanakkor továbbra is fontos, hogy a válaszok összetettebb feladatok esetén ellenőrzésre kerüljenek, ezért a későbbi fejlesztések egyik iránya a szélesebb körű tesztelés és a válaszmínőség további javítása lehet.

7. Mesterséges intelligencia használata a fejlesztés során

A projekt fejlesztése és dokumentálása során mesterséges intelligencia alapú eszközöket is használtam. Elsősorban a ChatGPT-t és a Codex-et alkalmaztam, amelyek a fejlesztési folyamat különböző részeiben támogató szerepet tölthettek be. Ezeket az eszközöket nem önálló fejlesztőként kezeltem, hanem olyan segédeszközként, amely ötleteket, magyarázatokat, javítási javaslatokat és szerkesztési támogatást adott. A végső döntéseket, a módosítások elfogadását, a kód ellenőrzését és a működés validálását minden esetben saját ellenőrzés alapján végeztem.

Az AI-eszközöket többféle feladatra használtam. A fejlesztés korábbi szakaszaiban segítséget nyújtottak az architektúra átgondolásában, a frontend, backend, adatbázis és AI tutor modul szerepének elkülönítésében, valamint a dokumentációs struktúra kialakításában. Emellett hasznosak voltak a Markdown dokumentációk megfogalmazásában, például az API-végpontok, az adatmodell, a reprodukálható futtatás, a tesztelés és az AI tutor működésének leírásánál. Ezeknél az AI által adott javaslatokat mindig a projekt tényleges kódjához és működéséhez igazítottam.

A kódfejlesztés során az AI-eszközöket főként magyarázatkérésre, hibakeresésre és kisebb javítási ötletek kidolgozására használtam. Például a backend autentikációs működésének, a chatfolyamatnak és az AI tutor modul viselkedésének értelmezésében is segítséget adtak. Az AI tutor esetében különösen fontos volt a modellválaszok kezelése, mert a nyelvi modell nem minden esetben adott pontosan olyan formátumú választ, amelyet a rendszer elvárt. A feladatgenerálás validációja során például előfordult, hogy a modell nem közvetlenül matematikai feladatot adott vissza, hanem rövid meta jellegű szöveget kezdett generálni arról, hogy milyen feladatot fog létrehozni. Ezt a hibát a rendszerutasítás és a válasz-utófeldolgozás pontosításával javítottam, majd a működést újra teszteltem.

Az AI által adott javaslatokat nem tekintettem automatikusan helyesnek. Minden fontosabb javaslatot ellenőriztem a projekt forráskódja, a futtatott alkalmazás és a dokumentáció alapján. Ha az AI olyan megoldást javasolt, amely nem illeszkedett a tényleges kódbázishoz, akkor azt módosítottam vagy elvetettem. A validáció egyik konkrét példája a feladatgenerálási működés javítása volt, az első teszt során a tutor válasza nem felelt meg teljesen az elvárásnak, a módosítás után viszont már egy konkrét

lineáris egyenletet generált megoldás nélkül. Ez jól mutatta, hogy az AI által támogatott fejlesztésnél a tesztelés és az emberi ellenőrzés elengedhetetlen.

Az AI-eszközök a szakdolgozat szövegének elkészítésében is szerepet kaptak. Elsősorban a fejezetek szerkezetének átgondolásában, a megfogalmazások javításában és a szöveg gördülékenyebbé tételében segítettek. Fontos szempont volt, hogy az egyes fejezetek ne ugyanazt ismételjék meg, hanem eltérő nézőpontból mutassák be a rendszert. Például a technológiai fejezetben a felhasznált eszközök szerepe, a rendszertervben a komponensek kapcsolata, a megvalósítási fejezetben pedig a konkrét működés került előtérbe. A szakdolgozat gondolati felépítését, a saját projekt bemutatását és a végleges tartalmi döntéseket azonban nem adtam át teljesen az AI-nak.

Voltak olyan részek is, ahol tudatosan nem használtam mesterséges intelligenciát vagy csak korlátozottan támaszkodtam rá. Ilyen volt például a futó alkalmazás kézi kipróbálása, a tesztek tényleges végrehajtása, a képernyőképek elkészítése, valamint annak eldöntése, hogy egy adott működés elfogadható-e a szakdolgozat céljai szempontjából. Az AI nem férhetett hozzá titkos kulcsokhoz, valódi jelszavakhoz vagy érzékeny konfigurációs értékekhez. Ezeket az adatokat nem adtam meg bemenetként, a dokumentációban pedig csak általános vagy példaként szolgáló környezeti változók szerepelnek.

Az AI használatának egyik előnye az volt, hogy gyorsította a hibakeresést és segített többféle megoldási lehetőséget átgondolni. Hasznos volt akkor is, amikor egy dokumentációs rész vagy szakdolgozati fejezet szerkezetét kellett tisztázni. Ugyanakkor az is láthatóvá vált, hogy az AI-eszközök könnyen adhatnak túl általános, pontatlan vagy a konkrét projekthez nem teljesen illeszkedő választ. Emiatt minden javaslatot ellenőrizni kellett, különösen akkor, ha az kódmódosítást, rendszertervet vagy tesztelési eredményt érintett.

Összességében a mesterséges intelligencia a fejlesztési folyamatot támogató eszközként jelent meg. Segített a tervezésben, a hibakeresésben, a dokumentáció egységesítésében és a szakdolgozati szöveg szerkesztésében, de nem váltotta ki a fejlesztői döntéshozatalt és a validációt. A projekt egyik fontos tanulsága az volt, hogy az AI hatékonyan tudja támogatni a munkát, de csak akkor használható megbízhatóan, ha a kapott javaslatokat folyamatos ellenőrzés, tesztelés és saját szakmai döntés követi.

8. Összegzés és továbbfejlesztés

A dolgozat célja egy olyan webalkalmazás megtervezése és megvalósítása volt, amely mesterséges intelligencia segítségével támogatja a matematikai feladatok gyakorlását és megértését. A fejlesztés eredményeként elkészült egy bejelentkezéshez kötött rendszer, amelyben a felhasználó matematikai kérdéseket tehet fel, gyakorló feladatokat kérhet, folytathatja korábbi beszélgetéseit, valamint mappákba rendezheti az előzményeit.

A rendszer frontendből, backendből, adatbázisból és AI tutor modulból épül fel. A frontend biztosítja a felhasználói felületet, a backend kezeli az autentikációt, az API-végpontokat, az adatbázisműveleteket és az AI tutor meghívását, az adatok pedig PostgreSQL adatbázisban tárolódnak. A projekt Docker Compose segítségével reprodukálható módon futtatható, ami támogatja az alkalmazás ellenőrizhetőségét és későbbi továbbfejlesztését.

Az elkészült alkalmazás egyik központi eleme az AI tutor modul, amely Qwen-alapú nyelvi modellre épül. A tutor célja nem az, hogy minden esetben azonnal teljes megoldást adjon, hanem az, hogy matematikai kérdések esetén rövid, tanulást támogató választ vagy következő lépést fogalmazzon meg. A rendszer külön kezeli a feladatgenerálást, a problémamegoldást, a folytató kérdéseket, a válaszellenőrzést és a nem matematikai kérések elutasítását.

A tesztelés és validáció során ellenőrzésre kerültek a rendszer fő funkciói. A felhasználókezelés, a beszélgetések indítása és folytatása, a mappakezelés, valamint a profilhoz kapcsolódó műveletek működőképesnek bizonyultak. Az AI tutor manuális validációja során a rendszer matematikai kérdésekre tutor jellegű választ adott, a nem matematikai kéréseket pedig megfelelően elutasította. A feladatgenerálásnál kezdetben tapasztalt pontatlanság javításra került, így a rendszer a módosítás után konkrét matematikai feladatot adott vissza megoldás nélkül.

A fejlesztés során több korlát is azonosítható volt. Az AI-alapú válaszadás miatt a generált válaszok nem tekinthetők minden esetben hibátlannak, ezért összetettebb matematikai feladatoknál továbbra is szükséges lehet az eredmények ellenőrzése. A modell futtatása erőforrás-igényes lehet, különösen GPU nélküli környezetben. Emellett a rendszer további fejlesztést igényelhet mobilos használhatóság, válaszminőség, tesztelési lefedettség és éles üzemi biztonság szempontjából.

Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

Továbbfejlesztési lehetőségként megemlíthető a válaszok részletesebb automatikus értékelése, a streaming válaszáadás bevezetése, a mobilos felület finomítása, valamint a matematikai témakörök célzottabb kezelése. Hasznos irány lehet továbbá a tutor válaszainak szélesebb tesztalacson történő vizsgálata és a felhasználói élmény további javítása.

Össességében a dolgozatban bemutatott rendszer teljesíti a kitűzött alapvető célokat. Elkészült egy működő webalkalmazás, amely felhasználói fiókkezeléssel, mentett beszélgetésekkel, mappakezeléssel és AI-alapú matematikai tutor funkcióval támogatja a tanulási folyamatot. A projekt megfelelő alapot biztosít egy továbbfejleszthető, célzott matematikai tanulástámogató alkalmazás kialakításához.

Irodalomjegyzék

1. Visual Studio Code: <https://code.visualstudio.com/> (Utolsó megtekintés: 2026. 04. 21.)
2. Next.js: <https://nextjs.org/> (Utolsó megtekintés: 2026. 04. 21.)
3. TypeScript: <https://www.typescriptlang.org/> (Utolsó megtekintés: 2026. 04. 21.)
4. FastAPI: <https://fastapi.tiangolo.com/> (Utolsó megtekintés: 2026. 04. 21.)
5. PostgreSQL: <https://www.postgresql.org/> (Utolsó megtekintés: 2026. 04. 21.)
6. Docker: <https://www.docker.com/> (Utolsó megtekintés: 2026. 04. 21.)
7. Qwen2.5: <https://huggingface.co/papers/2412.15115> (Utolsó megtekintés: 2026. 04. 21.)
8. Python: <https://www.python.org/> (Utolsó megtekintés: 2026. 04. 21.)
9. OECD. PISA 2022 Results (Volume I): The State of Learning and Equity in Education. OECD Publishing, Paris, 2023.:
https://www.oecd.org/en/publications/pisa-2022-results-volume-i_53f23881-en.html (Utolsó megtekintés: 2026. 04. 21.)
10. Khanmigo: <https://www.khanmigo.ai/> (Utolsó megtekintés: 2026. 04. 25.)
11. Quizlet AI: <https://quizlet.com/features/ai-study-tools> (Utolsó megtekintés: 2026. 04. 25.)
12. ChatGPT Edu: <https://openai.com/index/introducing-chatgpt-edu/> (Utolsó megtekintés: 2026. 04. 25.)
13. Photomath: <https://photomath.com/> (Utolsó megtekintés: 2026. 04. 25.)
14. Socratic by Google: <https://support.google.com/socratic/?hl=en#topic=9414097> (Utolsó megtekintés: 2026. 04. 25.)
15. Microsoft Math Solver: <https://mathsolver.microsoft.com/> (Utolsó megtekintés: 2026. 04. 25.)
16. Wolfram Alpha: <https://www.wolframalpha.com/> (Utolsó megtekintés: 2026. 04. 25.)
17. Tailwind CSS: <https://tailwindcss.com/> (Utolsó megtekintés: 2026. 04. 25.)
18. Qwen2.5-Math Technical Report: <https://arxiv.org/abs/2409.12122> (Utolsó megtekintés: 2026. 04. 25.)
19. Hugging Face Transformers: <https://huggingface.co/docs/transformers/index> (Utolsó megtekintés: 2026. 04. 25.)
20. React: <https://react.dev/learn> (Utolsó megtekintés: 2026. 04. 25.)
21. MDN Web Docs - JavaScript: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (Utolsó megtekintés: 2026. 04. 25.)
22. React Hot Toast: <https://react-hot-toast.com/> (Utolsó megtekintés: 2026. 04. 25.)
23. KaTeX: <https://katex.org/docs/> (Utolsó megtekintés: 2026. 04. 25.)
24. Rest API: <https://www.ibm.com/think/topics/rest-apis> (Utolsó megtekintés: 2026. 04. 25.)
25. Jones, M., Bradley, J., Sakimura, N.: JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force, 2015.: <https://datatracker.ietf.org/doc/html/rfc7519> (Utolsó megtekintés: 2026. 05. 12.)
26. SQLAlchemy: <https://docs.sqlalchemy.org/en/20/> (Utolsó megtekintés: 2026. 05. 12.)
27. PyTorch: <https://pytorch.org/docs/stable/index.html> (Utolsó megtekintés: 2026. 05. 12.)
28. bitsandbytes: <https://huggingface.co/docs/bitsandbytes/index> (Utolsó megtekintés: 2026. 05. 12.)
29. Pydantic: <https://pydantic.dev/> (Utolsó megtekintés: 2026. 05. 12.)
30. GitHub: <https://www.github.com/> (Utolsó megtekintés: 2026. 05. 20.)
31. JSON: <https://www.json.org/json-en.html> (Utolsó megtekintés: 2026. 05. 20.)

Nyilatkozat

Alulírott Sztricskó Zoltán Milán, programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026. 05. 20.

Aláírás

Köszönetnyilvánítás

Szeretnék köszönetet mondani konzulensemnek, Dr. Bilicki Vilmosnak, aki szakmai útmutatásával és hasznos visszajelzéseivel segítette a dolgozat elkészítését.

Külön köszönettel tartozom a családomnak, akik támogatásukkal, türelmükkel és biztatásukkal végig mellettem álltak. Köszönöm a barátaimnak is, hogy a dolgozat készítése során bátorítottak, meghallgattak és segítettek abban, hogy a nehezebb időszakokon is tovább tudjak haladni.

Elektronikus mellékletek

1. A webalkalmazás GitHub elérhetősége: <https://github.com/tyotyesz/ai-mathstudent-assistant>
(Utolsó megtekintés: 2026. 05. 21.)

Mellékletek

Azonosító	Követelmény	Rövid leírás
FK-01	Regisztráció	A felhasználó e-mail cím és jelszó megadásával fiókot hozhat létre.
FK-02	Bejelentkezés	A felhasználó a regisztrált adataival beléphet az alkalmazásba.
FK-03	Új beszélgetés indítása	A felhasználó új matematikai tutor beszélgetést kezdhet.
FK-04	Beszélgetés folytatása	A felhasználó meglévő beszélgetéshez új üzenetet küldhet.
FK-05	Beszélgetések mentése	A rendszer eltárolja a felhasználó beszélgetéseit.
FK-06	Beszélgetések megnyitása	A felhasználó visszatérhet korábbi beszélgetéseihez.
FK-07	Mappakezelés	A felhasználó mappákat hozhat létre és beszélgetéseket rendezhet beléjük.
FK-08	Profilkezelés	A felhasználó megtekintheti profilját, módosíthatja jelszavát vagy törölheti fiókját.
FK-09	Matematikai tutor válaszadás	A rendszer AI-alapú választ ad matematikai jellegű kérdésekre.
FK-10	Nem matematikai kérések kezelése	A rendszer elutasítja a nem matematikai témájú kéréseket.

M2.3.1.ábra: A rendszer fő funkcionális követelményei

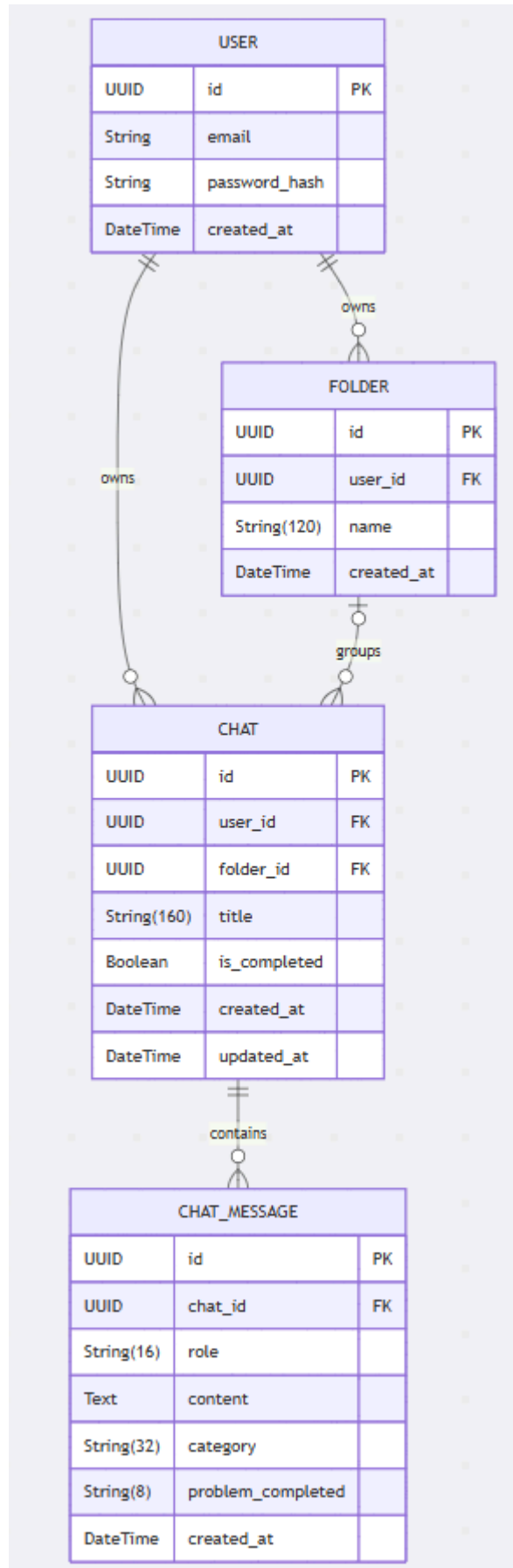
Mesterséges Intelligencia alapú diákasszisztens webalkalmazás matematikai feladatok támogatására

Terület	Követelmény
<i>Használhatóság</i>	A fő funkciók legyenek könnyen elérhetők és érthetők.
<i>Asztali megjelenés</i>	Az alkalmazás nagyobb kijelzőn legyen áttekinthetően használható.
<i>Alapszintű reszponzivitás</i>	A fő elrendezés bizonyos elemei kisebb kijelzőn is alkalmazkodjanak.
<i>Mobilos optimalizálás</i>	A részletes mobilos navigáció és finomhangolás továbbfejlesztési lehetőségként jelenik meg.
<i>Hibakezelés</i>	A rendszer adjon egyértelmű visszajelzést hibás vagy sikertelen művelet esetén.
<i>Biztonság</i>	A felhasználói adatok csak bejelentkezés után legyenek elérhetők.
<i>Jogosultságkezelés</i>	A felhasználó csak a saját beszélgetéseéhez, mappáihoz és profiladataihoz férjen hozzá.
<i>Reprodukálhatóság</i>	A rendszer dokumentált lépések alapján, Docker segítségével elindítható legyen.
<i>Konfigurálhatóság</i>	A szükséges beállítások környezeti változókon keresztül legyenek kezelhetők.
<i>AI-korlátok kezelése</i>	Az AI-válaszok minőségét ellenőrizni kell, és a modellhibákat kezelni kell.

M2.4.1.ábra: A rendszer fő nem funkcionális követelményei

Könyvtár	Fontosság
FastAPI	Backend API-végpontok és szerveroldali működés kezelése
Uvicorn	A FastAPI alkalmazás futtatása
Pydantic	Bemeneti adatok és válaszmodellek validálása
SQLAlchemy	Adatbázismodellek és ORM-alapú adatkezelés
psycopg2	PostgreSQL adatbáziskapcsolat biztosítása
passlib, bcrypt	Jelszóhashelés kezelése
python-jose	JWT tokenek létrehozása és ellenőrzése
Transformers	A Qwen modell betöltése és futtatása
PyTorch	A nyelvi modell futtatásának mélytanulási alapja
huggingface_hub	Hugging Face modellek elérése
bitsandbytes	GPU esetén 4 bites kvantálás támogatása
pytest	Tesztek futtatása

M3.4.1.ábra: A backendhez és AI modulhoz kapcsolódó fő Python könyvtárak



M4.3.1.ábra: A rendszer fő adatmodellje